Hostel Complaint & Attender Duty Tracking System

Phase 1 — Design Document

Ajoy Mathew (CS23B101) CS5013 — Programming with AI, IIT Madras

11 September 2026

Contents

1. Overview & Scope **3**
1.1 Problem . . . 3
1.2 Goals . . . 3
1.3 Non-goals . . . 4
1.4 Actors . . . 4
1.5 Glossary . . . 4
1.6 Non-functional requirements . . . 5
1.7 Open questions . . . 6
1.8 Constraints . . . 6

2. System Architecture **7**
2.1 Architectural style . . . 7
2.2 Context diagram . . . 7
2.3 Module breakdown . . . 7
2.4 Layering rules . . . 9
2.5 The pipeline . . . 9
2.6 Deployment topology . . . 10
2.7 Cross-cutting concerns . . . 10

3. Data Model **12**
3.1 Design principles . . . 12
3.2 Entity–relationship diagram . . . 12
3.3 Identity & access . . . 12
3.4 Locations, coverage & availability . . . 13
3.5 Ingestion & complaints . . . 14
3.6 Tasks, dockets and the ledger . . . 15
3.7 Audit, outbox and classifier data . . . 16
3.8 Portability constraints . . . 17
3.9 Retention . . . 18

4. State Machines **19**
4.1 Why this is formalised . . . 19
4.2 Complaint lifecycle . . . 19
4.3 CMS docket sub-state . . . 21
4.4 Task lifecycle (special duties) . . . 22
4.5 Priority and aging . . . 22

5. Ingestion & Categorization **23**
5.1 The hard part . . . 23
5.2 Column mapping . . . 23

5.3 Delta detection . . . 23
5.4 Normalization to a complaint . . . 25
5.5 Categorization . . . 25
5.6 Meeting the 90% accuracy criterion . . . 26
5.7 OpenNLP as a deferred second stage . . . 26
5.8 CSV fallback . . . 26

6. Dispatch Heuristic **28**
6.1 Problem statement . . . 28
6.2 Formulation . . . 28
6.3 Shared tasks . . . 30
6.4 Degenerate cases . . . 30
6.5 Why greedy, and not optimal assignment . . . 30

7. Points Ledger & Concurrency Control **31**
7.1 What can actually go wrong . . . 31
7.2 Mechanisms . . . 31
7.3 Why double-crediting is structurally impossible . . . 31
7.4 Idempotency at the HTTP boundary . . . 32
7.5 Shared tasks . . . 32
7.6 Claiming a duty (C4) . . . 33
7.7 Optimistic locking and conflict UX . . . 33
7.8 The leaderboard . . . 33
7.9 Month boundaries . . . 34

8. Analytics & Telemetry **35**
8.1 Where the numbers come from . . . 35
8.2 MTTR and bottleneck detection . . . 35
8.3 Anomaly detection . . . 36
8.4 Dashboard visualizations . . . 37
8.5 Explicit limitations . . . 37

9. API & Data Contracts **38**
9.1 Shape of the web layer . . . 38
9.2 Conventions . . . 38
9.3 Attender API (mobile) . . . 38
9.4 Manager API . . . 40
9.5 SSE contract . . . 40
9.6 Push notification contract . . . 41
9.7 Error model . . . 42

10. Frontend Design **44**
10.1 Two clients, two very different users . . . 44
10.2 Attender mobile . . . 44
10.3 Manager desktop . . . 45
10.4 Screens . . . 46
10.5 Styling & accessibility . . . 47
10.6 Performance . . . 47

11. Security & Privacy **48**
11.1 Threat model . . . 48
11.2 Authentication . . . 48
11.3 Authorization . . . 49
11.4 PII and data minimisation . . . 49
11.5 Secrets . . . 50
11.6 Application security . . . 50

11.7 Audit as a security control . . . 50

12. Test Plan **51**
12.1 Strategy . . . 51
12.2 Fixtures . . . 51
12.3 Categorization gold set . . . 51
12.4 Corner-case suite . . . 52
12.5 Concurrency suite . . . 53
12.6 Acceptance-criteria traceability . . . 54
12.7 What we are not testing . . . 54

13. Revised Timeline **55**
13.1 Fixed dates . . . 55
13.2 Sprint plan . . . 55
13.3 Sprint detail . . . 55
13.4 Change from the original proposal . . . 57
13.5 Risk buffer and cut lines . . . 57

14. Decision Record **59**
ADR-001 · PostgreSQL everywhere, not SQLite/H2 for development . . . 59
ADR-002 · Local credentials, invite-only; no Google OAuth . . . 59
ADR-003 · SSE, not WebSocket, for live updates . . . 59
ADR-004 · Rule-based lexicon as the primary classifier; OpenNLP deferred . . . 60
ADR-005 · Greedy per-complaint dispatch, not global optimisation . . . 60
ADR-006 · Pessimistic locking for duty claims, optimistic elsewhere . . . 61
ADR-007 · `READ COMMITTED` isolation, not `SERIALIZABLE` . . . 61
ADR-008 · Append-only ledger; no stored point balance . . . 61
ADR-009 · Content-derived identity for source rows, not row index . . . 62
ADR-010 · Modular monolith, not microservices . . . 62
ADR-011 · Web Push with VAPID for attender notifications, not FCM or a native app . . . 62

1. Overview & Scope

1.1 Problem

The Hostel Manager at Cauvery Hostel, IIT Madras processes daily maintenance complaints by hand. Residents submit a Google Form; the manager copies each response into a spreadsheet, sorts it into a domain (Electrical, Civil, RO Plant), checks which attenders are on duty and which floors they cover, assigns the work verbally, chases inspection IDs, and at month end tallies points for bonus calculation. This costs **30–45 minutes every day** and is error-prone at every step.

Four attenders serve the hostel. Points accrue at **+2** per resolved complaint, custom values for special duties, and **−1** per logged infraction. All of it is currently reconciled by hand in a spreadsheet that is easily corrupted when edited from mobile devices.

1.2 Goals

G	Goal	Measured by
G1	Eliminate manual copy-paste from the Form response sheet	Zero manual transcription; no duplicate complaints across polls
G2	Auto-categorize complaints into the correct domain	≥90% accuracy on the labelled gold set (Sec. 12.3)
G3	Assign each complaint to the most appropriate attender automatically	Dispatch produces an assignment or an explicit triage reason, 100% of the time

G	Goal	Measured by
G4	Give attenders a usable mobile queue	Attender can log a CMS docket and resolve a ticket in ≤3 taps from the queue
G5	Make point accounting correct and auditable	No double-credit under concurrent updates; every point movement traceable to an actor and a cause
G6	Replace the end-of-month manual tally	One-click `.xlsx` export; leaderboard always live
G7	Surface operational insight the manager cannot currently see	MTTR per category, bottleneck stage, anomaly alerts
G8	Reach an attender when work is assigned, without them opening the app	Push notification delivered to the attender's phone for new and escalated assignments

1.3 Non-goals

Explicitly **out of scope** for this project, to keep the solo build tractable:

- **Resident-facing accounts.** Residents keep using the existing Google Form. They never log in.
- **Replacing the IITM EU CMS app.** We record its docket number; we do not integrate with or supersede it. There is no CMS API available to us.
- **Native mobile applications.** The attender interface is a responsive web view.
- **Multi-hostel tenancy.** Single-hostel deployment. The schema does not model a hostel dimension.
- **Payroll or bonus disbursement.** We compute points; money is handled outside the system.
- **A native mobile app store presence.** Push reaches attenders through the installable web app (Web Push), not a Play Store binary.

1.4 Actors

Actor	Count	Device	Authenticated	Responsibilities
Hostel Manager	1	Desktop (office), occasionally mobile	Yes — username + password	Oversight, triage of unclassified complaints, reassignment, penalties, special duties, exports, account provisioning
Attender	~4	Personal Android phone, hostel Wi-Fi / mobile data	Yes — per-user PIN + device token	Receive push alerts, view assigned queue, record CMS docket, mark resolved, claim special duties, view own standing
Resident	~hundreds	Any	**No**	Submits the Google Form. Never touches this system directly.
Scheduler	—	—	System	Polls the sheet, ages priorities, runs telemetry scans

1.5 Glossary

Term	Meaning
Complaint	A single maintenance request originating from one Google Form response
Task	The unit of work dispatched to attender(s). One complaint yields one task; special duties are tasks with no complaint
Domain	Category of work: `ELECTRICAL`, `CIVIL`, `RO_PLANT`, `NETWORK`, or `UNCLASSIFIED`
CMS docket	The tracking number issued by the IITM EU Campus Management System when an attender files the issue officially
Triage queue	Holding area for complaints the classifier could not confidently categorize, or that dispatch could not assign
Ledger entry	An immutable row recording a point movement, with actor, cause and effective date
Special duty	A manager-posted ad-hoc task with a custom point value, claimable by any attender
Penalty	A manager-logged infraction carrying a negative point value
SLA	Target resolution time for a complaint, by domain and priority
OCC	Optimistic concurrency control — version-checked updates that fail rather than silently overwrite

1.6 Non-functional requirements

ID	Requirement	Target	Rationale
NFR-1	Scale envelope	≤50 complaints/day, ≤10 concurrent users, ≤50k rows/year	Single hostel, 4 attenders. Sizing everything else follows from this.
NFR-2	Ingestion latency	New Form response visible in the manager dashboard within **5 minutes**	Poll interval, not a real-time requirement
NFR-3	Interactive response time	p95 < 500 ms for queue and dashboard reads	Attenders on mobile data in hostel corridors
NFR-4	Mobile payload	Attender queue initial load < 150 KB	Low-end phones, patchy connectivity
NFR-5	Correctness under concurrency	Zero double-credits, zero lost updates	Core integrity guarantee; see Sec. 7
NFR-6	Availability	Best-effort. Offline fallback jar must run with **no internet at all**	Free-tier PaaS; outage plan B from proposal Sec. 10
NFR-10	Push delivery	P1 assignment push dispatched within **30 s** of assignment; delivery itself is best-effort	The push service is outside our control, so the in-app queue stays the source of truth
NFR-7	Timezone	All business dates in **Asia/Kolkata**; all timestamps stored UTC	Month boundaries for the bonus cycle must be unambiguous
NFR-8	Auditability	Every state change and point movement is reconstructible	Bonus disputes must be resolvable from the log

ID	Requirement	Target	Rationale
NFR-9	Data retention	Complaint text retained 1 year; resident contact detail retained 90 days	Minimize PII exposure (Sec. 11.4)

1.7 Open questions

Pending confirmation from the stakeholder. Each has a working default so no design work is blocked.

Q	Question	Working default
OQ-1	Exact format of the IITM EU CMS docket number	Configurable regex, permissive default `^[A-Za-z0-9/-]{4,32}$`. Validation is a config property, not a code constant.
OQ-2	Per-domain SLA targets	Electrical 8h, RO Plant 12h, Civil 24h, Network 24h (working hours). See Sec. 8.2.
OQ-3	Are attender floor allocations fixed or rotating?	Modelled as time-bounded assignments so both work. See Sec. 3.4.
OQ-4	Exact Google Form column headers and question set	Ingestion maps columns by header name via config, tolerant of reordering. See Sec. 5.2.
OQ-5	Do attenders have domain specialisations, or is everyone a generalist?	Skill weight defaults to neutral; scoring supports specialisation if it exists. See Sec. 6.2.
OQ-6	Working-hours window for SLA aging	08:00–20:00 IST, seven days

1.8 Constraints

- **Solo developer**, roughly nine weeks, alongside coursework. Scope is tagged Core/Extended in Sec. 13.
- **Free-tier hosting.** Instances sleep when idle; this shapes the scheduler design (Sec. 2.5).
- **No CMS API.** Docket numbers are entered by hand — the system cannot verify them upstream.
- **Low-tech end users.** Attenders may have limited literacy in English and low-end phones. UI must lean on iconography, large tap targets and minimal text entry.

2. System Architecture

2.1 Architectural style

A **modular monolith** deployed as a single Spring Boot executable jar, organised internally into bounded modules with explicit interfaces between them.

This is a deliberate choice over microservices. At NFR-1 scale (≤50 complaints/day, 4 attenders) a distributed system would add network failure modes, deployment complexity and eventual-consistency problems in exchange for scalability we will never need. The modular boundaries below give us the separation-of-concerns benefit without the operational cost, and they are drawn so that a module *could* be extracted later if that ever became necessary.

Within each module we use **ports and adapters (hexagonal)** where — and only where — an external system is involved. This is not decoration: the ingestion port is what allows the Google Sheets adapter and the CSV fallback adapter (proposal Sec. 10, Plan B) to share one normalization, dedupe and categorization pipeline instead of drifting into two code paths.

2.2 Context diagram

```
graph LR
    R[Resident] -->|submits| GF[Google Form]
    GF -->|appends row| GS[(Google Sheet<br/>responses)]

    subgraph SYS["Hostel Complaint & Duty Tracking System"]
        APP[Spring Boot Application]
    end

    GS -->|Sheets API v4<br/>poll| APP
    CSV[CSV file] -.->|manual upload<br/>fallback| APP
    APP --> DB[(PostgreSQL)]

    M[Hostel Manager<br/>desktop] <-->|HTTPS + SSE| APP
    A[Attender<br/>mobile] <-->|HTTPS| APP
    APP -->|.xlsx| XL[Excel report]
    A -.->|files issue manually,<br/>gets docket no.| CMS[IITM EU CMS App]

    style SYS fill:#eef4ff,stroke:#3b6bb5
    style CMS stroke-dasharray: 5 5
```

The dashed edge to the CMS app is important: there is **no programmatic integration**. The attender files the complaint there by hand and types the resulting docket number into our system. We store it for traceability but can never verify it upstream (see OQ-1).

2.3 Module breakdown

```
graph TD
    subgraph WEB["Web layer"]
        MC[Manager controllers]
        AC[Attender controllers]
        SSE[SSE broadcaster]
        PUSH[Web Push sender]
    end

    subgraph CORE["Domain modules"]
        ING[ingestion]
        CAT[categorization]
        DIS[dispatch]
        WF[workflow<br/>state machine]
```

```
    LED[ledger]
    ANA[analytics]
    NOT[notification]
    RPT[reporting]
    IAM[identity]
end

subgraph INFRA["Infrastructure"]
    REPO[JPA repositories]
    SHEETS[Sheets adapter]
    CSVA[CSV adapter]
    POI[POI writer]
    SCHED[Scheduler]
end

MC --> WF & DIS & LED & ANA & RPT & IAM
AC --> WF & LED & IAM
ING --> CAT --> DIS --> WF
WF --> LED
WF --> NOT --> SSE
NOT --> PUSH --> WPS[Browser push service]
LED --> NOT
ANA --> REPO
RPT --> POI
ING --> SHEETS & CSVA
SCHED --> ING & ANA & WF
CORE --> REPO --> PG[(PostgreSQL)]
```

Module	Java package	Responsibility	Key invariant it owns
ingestion	`...ingestion`	Fetch raw rows, detect deltas, normalize into complaints	A source row is never turned into two complaints
categorization	`...categorization`	Assign a domain and confidence to complaint text	Low confidence never silently becomes a wrong domain
dispatch	`...dispatch`	Score attenders, produce assignments	Every complaint gets an assignment *or* a recorded triage reason
workflow	`...workflow`	Complaint/task state machine, guards, transitions	Only legal transitions occur; transitions are atomic with their side effects
ledger	`...ledger`	Point movements, leaderboard projection	Append-only; no double-credit; balance is derived, never stored
analytics	`...analytics`	MTTR, bottlenecks, anomaly detection	Read-only over the ledger and audit tables
notification	`...notification`	Fan-out of domain events to manager SSE streams **and attender Web Push**	Delivery is best-effort; never blocks a transaction
reporting	`...reporting`	`.xlsx` generation via Apache POI	Reports are point-in-time snapshots, reproducible from stored data

Module	Java package	Responsibility	Key invariant it owns
identity	`...identity`	Users, roles, PINs, device tokens, provisioning	No self-registration; every principal is pre-provisioned
audit	`...audit`	Append-only event log	Written in the same transaction as the change it records

Root package: `in.ac.iitm.hostel.duty`

2.4 Layering rules

```
web (controllers, DTOs, Thymeleaf)
  ↓ may call
application (services, transaction boundaries, orchestration)
  ↓ may call
domain (entities, value objects, state machine, scoring — no Spring, no JPA annotations leaking behaviour)
  ↑ implemented by
infrastructure (JPA repos, Sheets client, POI, SSE)
```

Enforced rules, checked in CI by ArchUnit:

1. `domain` depends on nothing outside itself and the JDK.
2. `web` never touches a repository directly — it goes through `application`.
3. Cross-module calls go through the target module's published service interface, never its internal classes or its repositories.
4. `@Transactional` lives in `application`, never in `web` or in a repository.

Rule 4 matters more than it looks: transaction boundaries are where our concurrency guarantees (Sec. 7) are enforced, so they must be in one predictable place.

2.5 The pipeline

```
sequenceDiagram
    participant S as Scheduler
    participant I as Ingestion
    participant C as Categorizer
    participant D as Dispatch
    participant W as Workflow
    participant N as Notification
    participant M as Manager UI

    S->>I: poll (every 3 min, with lock)
    I->>I: fetch rows → diff against watermark + hashes
    alt new rows found
        I->>C: classify(text)
        C-->>I: domain + confidence
        alt confidence ≥ threshold
            I->>D: dispatch(complaint)
            D-->>I: assignment or NO_CANDIDATE
        else below threshold
            I->>I: mark UNCLASSIFIED → triage queue
        end
        I->>W: create complaint in NEW / TRIAGE
        W->>N: ComplaintCreated event
        N->>M: SSE push
```

```
    else nothing new
        I->>I: no-op, update poll timestamp only
    end
```

Scheduler safety. Free-tier PaaS instances sleep when idle, so `@Scheduled` polling cannot be relied on alone. Three mitigations:

1. A **"Sync Now"** button in the manager dashboard triggers ingestion on demand — this is the guaranteed path, and it also wakes a sleeping instance.
2. On startup the app runs a **catch-up ingestion**, so a wake after sleep loses nothing. Because the delta algorithm (Sec. 5.3) is watermark-and-hash based rather than time based, a long gap is handled identically to a short one.
3. Scheduled jobs take a **database-backed lock** (ShedLock) before running, so if the platform ever runs two instances they cannot double-ingest.

2.6 Deployment topology

```
graph TB
    subgraph CLOUD["Cloud — Render / Railway free tier"]
        JAR[app.jar<br/>embedded Tomcat :8080]
        PGC[(Managed PostgreSQL)]
        JAR --> PGC
    end

    subgraph LOCAL["Fallback — manager's desktop, offline capable"]
        JARL[app.jar<br/>localhost:8080]
        H2[(H2 file DB)]
        JARL --> H2
    end

    PHONE[Attender phones] -->|HTTPS| JAR
    DESK[Manager desktop] -->|HTTPS| JAR
    DESK -.->|during outage| JARL
    PGC -.->|nightly dump| BAK[Backup .sql]
    BAK -.->|seed| H2
```

Concern	Cloud (primary)	Local (fallback)
Database	Managed PostgreSQL	H2 file mode
Sheets ingestion	Active, polled	Disabled — CSV upload only
Authentication	Local credentials	Local credentials — **works with no internet**, which is why OAuth was rejected (ADR-002)
Purpose	Day-to-day operation	Business continuity during an outage

The fallback profile is why D1 keeps H2 in the picture at all. The schema is deliberately constrained to the **portable subset** of SQL that both engines support (Sec. 3.7), and Flyway migrations carry no vendor-specific DDL.

2.7 Cross-cutting concerns

Concern	Approach
Configuration	Spring profiles: `local`, `cloud`, `offline`. All tunables (dispatch weights, thresholds, SLA table, poll interval, CMS regex) are `@ConfigurationProperties`, never constants.

Concern	Approach
Secrets	Environment variables only. The Google service-account JSON is supplied as a base64 env var, never a committed file. `.gitignore` blocks the obvious filenames (Sec. 11.5).
Migrations	Flyway, forward-only, versioned. Schema changes never hand-edited.
Errors	`@ControllerAdvice` maps domain exceptions to the RFC 7807 problem envelope (Sec. 9.7).
Observability	Spring Actuator health/metrics; structured logging with a correlation ID per request and per ingestion run.
Idempotency	Every state-changing attender action carries a client-generated request key (Sec. 7.4).

3. Data Model

3.1 Design principles

1. **Facts are immutable; state is derived.** Point balances are never stored as a mutable counter. They are the sum of an append-only ledger. This makes double-crediting a constraint violation rather than a race we have to win (Sec. 7).
2. **Every mutable aggregate root carries a `version` column** for optimistic locking.
3. **Audit is written inside the same transaction as the change**, so the log can never disagree with the data.
4. **No hard deletes** on operational data. Rows are tombstoned so the audit trail stays intact.
5. **Portable SQL only.** DDL must run on both PostgreSQL and H2 (Sec. 3.7).

3.2 Entity–relationship diagram

```
erDiagram
    APP_USER ||--o{ DEVICE_TOKEN : "authenticates via"
    APP_USER ||--o{ FLOOR_ASSIGNMENT : "covers"
    APP_USER ||--o{ AVAILABILITY_WINDOW : "declares"
    APP_USER ||--o{ TASK_ASSIGNMENT : "works"
    APP_USER ||--o{ POINT_LEDGER : "earns"

    FLOOR ||--o{ FLOOR_ASSIGNMENT : "assigned in"
    FLOOR ||--o{ COMPLAINT : "located on"

    SOURCE_ROW ||--o| COMPLAINT : "normalizes to"
    INGESTION_RUN ||--o{ SOURCE_ROW : "observed"

    COMPLAINT ||--o| TASK : "yields"
    COMPLAINT ||--o{ CMS_DOCKET : "filed as"
    TASK ||--o{ TASK_ASSIGNMENT : "allocated to"
    TASK ||--o{ POINT_LEDGER : "credits"

    APP_USER ||--o{ AUDIT_EVENT : "acted"
    CATEGORY_TERM }o--|| DOMAIN_REF : "scores toward"
    OUTBOX_EVENT }o--|| APP_USER : "notifies"
```

3.3 Identity & access

```sql
CREATE TABLE app_user (
    id              BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    username        VARCHAR(64)  NOT NULL,
    display_name    VARCHAR(128) NOT NULL,
    role            VARCHAR(16)  NOT NULL,   -- MANAGER | ATTENDER
    password_hash   VARCHAR(72),             -- BCrypt; MANAGER only
    pin_hash        VARCHAR(72),             -- BCrypt; ATTENDER only
    pin_set_at      TIMESTAMP,
    must_reset_pin  BOOLEAN      NOT NULL DEFAULT FALSE,
    failed_attempts INTEGER      NOT NULL DEFAULT 0,
    locked_until    TIMESTAMP,
    active          BOOLEAN      NOT NULL DEFAULT TRUE,
    created_at      TIMESTAMP    NOT NULL,
    created_by      BIGINT       REFERENCES app_user(id),
    version         BIGINT       NOT NULL DEFAULT 0,
    CONSTRAINT uq_user_username UNIQUE (username),
    CONSTRAINT ck_user_role     CHECK (role IN ('MANAGER','ATTENDER')),
    CONSTRAINT ck_user_cred     CHECK (
        (role = 'MANAGER'  AND password_hash IS NOT NULL) OR
        (role = 'ATTENDER' AND pin_hash      IS NOT NULL))
);
```

```sql
CREATE TABLE device_token (
    id           BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    user_id      BIGINT       NOT NULL REFERENCES app_user(id),
    selector     VARCHAR(32)  NOT NULL,   -- lookup half, indexed
    token_hash   VARCHAR(64)  NOT NULL,   -- SHA-256 of the validator half
    label        VARCHAR(64),             -- "Ravi's phone"
    issued_at    TIMESTAMP    NOT NULL,
    expires_at   TIMESTAMP    NOT NULL,
    last_used_at TIMESTAMP,
    revoked_at   TIMESTAMP,
    CONSTRAINT uq_device_selector UNIQUE (selector)
);
CREATE INDEX ix_device_user ON device_token (user_id, revoked_at);
```

There is **no self-registration path**. `app_user` rows are created only by a `MANAGER` through the provisioning screen, which is the enforcement point for "only a specific set of accounts may access the system". Because credentials are local, an attender needs no institute account and no Google account — the requirement to support non-institute people is met by *removing* an external dependency rather than adding one (ADR-002).

The split-token scheme (`selector` + hashed `validator`) is used rather than a single opaque token so that lookup is an indexed equality match while the secret half is still only stored hashed. A stolen database therefore does not yield usable device cookies.

3.4 Locations, coverage & availability

```sql
CREATE TABLE floor (
    id           BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    wing         VARCHAR(16)  NOT NULL,
    floor_number INTEGER      NOT NULL,
    label        VARCHAR(64)  NOT NULL,
    CONSTRAINT uq_floor UNIQUE (wing, floor_number)
);

-- Time-bounded so that both fixed and rotating allocations are expressible (OQ-3).
CREATE TABLE floor_assignment (
    id           BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    attender_id  BIGINT      NOT NULL REFERENCES app_user(id),
    floor_id     BIGINT      NOT NULL REFERENCES floor(id),
    coverage     VARCHAR(16) NOT NULL,   -- PRIMARY | SECONDARY
    valid_from   DATE        NOT NULL,
    valid_to     DATE,                   -- NULL = open ended
    version      BIGINT      NOT NULL DEFAULT 0,
    CONSTRAINT ck_coverage CHECK (coverage IN ('PRIMARY','SECONDARY')),
    CONSTRAINT ck_validity CHECK (valid_to IS NULL OR valid_to >= valid_from)
);
CREATE INDEX ix_fa_lookup ON floor_assignment (floor_id, valid_from, valid_to);

CREATE TABLE availability_window (
    id          BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    attender_id BIGINT       NOT NULL REFERENCES app_user(id),
    status      VARCHAR(16)  NOT NULL,   -- AVAILABLE | ON_LEAVE | OFF_DUTY
    starts_at   TIMESTAMP    NOT NULL,
    ends_at     TIMESTAMP,
    reason      VARCHAR(255),
    recorded_by BIGINT       NOT NULL REFERENCES app_user(id),
    CONSTRAINT ck_avail_status CHECK (status IN ('AVAILABLE','ON_LEAVE','OFF_DUTY'))
);
CREATE INDEX ix_avail_lookup ON availability_window (attender_id, starts_at, ends_at);
```

Modelling floor coverage as **time-bounded assignments** rather than a column on the attender means we do not have to know today whether allocations rotate (OQ-3) — and, critically, it means a dispatch decision made last month can still be explained using the coverage that was in force *then*, which a mutable column would have destroyed.

3.5 Ingestion & complaints

```
CREATE TABLE ingestion_run (
    id            BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    source_type   VARCHAR(16) NOT NULL,   -- GOOGLE_SHEETS | CSV_UPLOAD
    trigger_type  VARCHAR(16) NOT NULL,   -- SCHEDULED | MANUAL | STARTUP
    started_at    TIMESTAMP   NOT NULL,
    finished_at   TIMESTAMP,
    rows_seen     INTEGER     NOT NULL DEFAULT 0,
    rows_created  INTEGER     NOT NULL DEFAULT 0,
    rows_amended  INTEGER     NOT NULL DEFAULT 0,
    rows_vanished INTEGER     NOT NULL DEFAULT 0,
    status        VARCHAR(16) NOT NULL,   -- RUNNING | SUCCESS | FAILED
    error_detail  VARCHAR(2000),
    triggered_by  BIGINT REFERENCES app_user(id)
);

-- The idempotency anchor for ingestion. One row per observed source record.
CREATE TABLE source_row (
    id             BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    source_type    VARCHAR(16)  NOT NULL,
    source_ref     VARCHAR(128) NOT NULL,  -- sheet id, or upload batch id
    response_key   CHAR(64)     NOT NULL,  -- SHA-256(timestamp || reporter email): STABLE identity
    content_hash   CHAR(64)     NOT NULL,  -- SHA-256 of all normalized cells: change detection
    row_index      INTEGER,                -- positional HINT only, never identity (see 5.3)
    raw_payload    TEXT         NOT NULL,  -- JSON string of the original row
    state          VARCHAR(16)  NOT NULL,  -- ACTIVE | AMENDED | VANISHED
    revision       INTEGER      NOT NULL DEFAULT 1,
    first_seen_run BIGINT       NOT NULL REFERENCES ingestion_run(id),
    last_seen_run  BIGINT       NOT NULL REFERENCES ingestion_run(id),
    first_seen_at  TIMESTAMP    NOT NULL,
    last_seen_at   TIMESTAMP    NOT NULL,
    complaint_id   BIGINT,
    CONSTRAINT uq_source_row UNIQUE (source_type, source_ref, response_key),
    CONSTRAINT ck_sr_state   CHECK (state IN ('ACTIVE','AMENDED','VANISHED'))
);
CREATE INDEX ix_sr_hash ON source_row (content_hash);

-- Amendment history: an edited Form response keeps its response_key but gains a new revision.
CREATE TABLE source_row_revision (
    id            BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    source_row_id BIGINT     NOT NULL REFERENCES source_row(id),
    revision      INTEGER    NOT NULL,
    content_hash  CHAR(64)   NOT NULL,
    raw_payload   TEXT       NOT NULL,
    observed_at   TIMESTAMP  NOT NULL,
    observed_run  BIGINT     NOT NULL REFERENCES ingestion_run(id),
    CONSTRAINT uq_srr UNIQUE (source_row_id, revision)
);

CREATE TABLE complaint (
    id                BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    public_ref        VARCHAR(24)  NOT NULL,   -- e.g. CAU-2026-0417, shown to humans
    source_row_id     BIGINT       REFERENCES source_row(id),
    raw_text          TEXT         NOT NULL,
    reported_at       TIMESTAMP    NOT NULL,
    reporter_room     VARCHAR(32),
    reporter_contact  VARCHAR(128),           -- purged after 90 days (NFR-9)
    floor_id          BIGINT       REFERENCES floor(id),
    location_note     VARCHAR(255),
    domain            VARCHAR(16)  NOT NULL,  -- ELECTRICAL|CIVIL|RO_PLANT|NETWORK|UNCLASSIFIED
    domain_confidence NUMERIC(4,3),           -- 0.000 .. 1.000
    classified_by     VARCHAR(16)  NOT NULL,  -- RULES | MANUAL | FALLBACK
    priority          VARCHAR(8)   NOT NULL,  -- P1 | P2 | P3
    base_priority     VARCHAR(8)   NOT NULL,  -- before aging, kept for explainability
    status            VARCHAR(24)  NOT NULL,
    sla_due_at        TIMESTAMP,
    sla_breached      BOOLEAN      NOT NULL DEFAULT FALSE,
    created_at        TIMESTAMP    NOT NULL,
    updated_at        TIMESTAMP    NOT NULL,
```

```
    closed_at         TIMESTAMP,
    version           BIGINT       NOT NULL DEFAULT 0,
    CONSTRAINT uq_complaint_ref UNIQUE (public_ref),
    CONSTRAINT ck_domain   CHECK (domain IN
        ('ELECTRICAL','CIVIL','RO_PLANT','NETWORK','UNCLASSIFIED')),
    CONSTRAINT ck_priority CHECK (priority IN ('P1','P2','P3')),
    CONSTRAINT ck_conf     CHECK (domain_confidence IS NULL
                                  OR (domain_confidence >= 0 AND domain_confidence <= 1))
);
CREATE INDEX ix_complaint_status ON complaint (status, priority, reported_at);
CREATE INDEX ix_complaint_domain ON complaint (domain, created_at);
CREATE INDEX ix_complaint_sla    ON complaint (sla_due_at, sla_breached);
```

`complaint.raw_text` keeps the resident's original wording verbatim. The classifier's output is stored *alongside* it with a confidence score and the method used, never in place of it — so a misclassification is always recoverable and the gold-set evaluation in Sec. 12.3 can be run retrospectively over real production data.

3.6 Tasks, dockets and the ledger

```
CREATE TABLE task (
    id           BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    task_type    VARCHAR(16) NOT NULL,     -- COMPLAINT | SPECIAL_DUTY
    complaint_id BIGINT REFERENCES complaint(id),
    title        VARCHAR(255) NOT NULL,
    description  TEXT,
    point_value  INTEGER     NOT NULL,     -- +2 default for complaints; custom for duties
    shared       BOOLEAN     NOT NULL DEFAULT FALSE,
    status       VARCHAR(24) NOT NULL,
    opens_at     TIMESTAMP,
    closes_at    TIMESTAMP,
    created_by   BIGINT      NOT NULL REFERENCES app_user(id),
    created_at   TIMESTAMP   NOT NULL,
    version      BIGINT      NOT NULL DEFAULT 0,
    CONSTRAINT ck_task_type CHECK (task_type IN ('COMPLAINT','SPECIAL_DUTY')),
    CONSTRAINT ck_task_link CHECK (
        (task_type = 'COMPLAINT'    AND complaint_id IS NOT NULL) OR
        (task_type = 'SPECIAL_DUTY' AND complaint_id IS NULL))
);

CREATE TABLE task_assignment (
    id           BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    task_id      BIGINT      NOT NULL REFERENCES task(id),
    attender_id  BIGINT      NOT NULL REFERENCES app_user(id),
    assign_method VARCHAR(16) NOT NULL,    -- AUTO | MANUAL | CLAIMED
    assigned_by  BIGINT      REFERENCES app_user(id),
    assigned_at  TIMESTAMP   NOT NULL,
    status       VARCHAR(16) NOT NULL,     -- ACTIVE | RELEASED | COMPLETED
    score_snapshot TEXT,                   -- JSON of the dispatch score that chose them
    version      BIGINT      NOT NULL DEFAULT 0,
    CONSTRAINT uq_task_attender UNIQUE (task_id, attender_id),
    CONSTRAINT ck_asg_method CHECK (assign_method IN ('AUTO','MANUAL','CLAIMED'))
);

CREATE TABLE cms_docket (
    id           BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    complaint_id BIGINT       NOT NULL REFERENCES complaint(id),
    docket_no    VARCHAR(64)  NOT NULL,
    docket_state VARCHAR(24)  NOT NULL,   -- see Sec. 4.3
    filed_by     BIGINT       NOT NULL REFERENCES app_user(id),
    filed_at     TIMESTAMP    NOT NULL,
    superseded_at TIMESTAMP,              -- non-null when corrected by a later entry
    note         VARCHAR(500),
    version      BIGINT       NOT NULL DEFAULT 0
);
CREATE INDEX ix_docket_complaint ON cms_docket (complaint_id, superseded_at);
CREATE INDEX ix_docket_no        ON cms_docket (docket_no);
```

A docket is a **child row, not a column on `complaint`**. Attenders mistype numbers on phones; the correction has to be recorded rather than overwrite the original, both for audit and because a docket number is the evidence of institutional filing. The current docket is the row with `superseded_at IS NULL`.

```
-- Append-only. No UPDATE, no DELETE. Reversals are new rows with negated points.
CREATE TABLE point_ledger (
    id              BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    attender_id     BIGINT       NOT NULL REFERENCES app_user(id),
    task_id         BIGINT       REFERENCES task(id),
    reason_code     VARCHAR(32)  NOT NULL,  -- see table below
    points          INTEGER      NOT NULL,
    effective_date  DATE         NOT NULL,  -- Asia/Kolkata business date (NFR-7)
    period_key      CHAR(7)      NOT NULL,  -- 'YYYY-MM', derived from effective_date
    note            VARCHAR(500),
    idempotency_key VARCHAR(128) NOT NULL,
    reverses_id     BIGINT       REFERENCES point_ledger(id),
    created_by      BIGINT       NOT NULL REFERENCES app_user(id),
    created_at      TIMESTAMP    NOT NULL,
    CONSTRAINT uq_ledger_idem   UNIQUE (idempotency_key),
    CONSTRAINT ck_ledger_reason CHECK (reason_code IN
        ('COMPLAINT_RESOLVED','SPECIAL_DUTY','PENALTY','MANUAL_ADJUSTMENT','REVERSAL'))
);
CREATE INDEX ix_ledger_period  ON point_ledger (period_key, attender_id);
CREATE INDEX ix_ledger_att     ON point_ledger (attender_id, effective_date);
```

`reason_code`	Points	Raised by	Idempotency key
`COMPLAINT_RESOLVED`	+2 (configurable)	Workflow, on transition to `RESOLVED`	`RESOLVE:{task_id}:{attender_id}`
`SPECIAL_DUTY`	`task.point_value`	Workflow, on special-duty completion	`DUTY:{task_id}:{attender_id}`
`PENALTY`	−1 (configurable)	Manager	`PENALTY:{uuid}` from the request
`MANUAL_ADJUSTMENT`	any	Manager, with mandatory note	`ADJ:{uuid}`
`REVERSAL`	negation of target	Manager	`REV:{reverses_id}`

This table is the core integrity mechanism. Because the key for an automatic credit is derived deterministically from the task and attender, a retried mobile request — the realistic failure mode on patchy hostel Wi-Fi — collides on `uq_ledger_idem` and is swallowed as a no-op. Double-crediting is not a race we have to win; it is structurally impossible. See Sec. 7.3 for the full argument.

3.7 Audit, outbox and classifier data

```
CREATE TABLE audit_event (
    id             BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    occurred_at    TIMESTAMP   NOT NULL,
    actor_id       BIGINT REFERENCES app_user(id),  -- NULL = system
    actor_role     VARCHAR(16),
    entity_type    VARCHAR(32) NOT NULL,
    entity_id      BIGINT      NOT NULL,
    event_type     VARCHAR(48) NOT NULL,
    from_state     VARCHAR(24),
    to_state       VARCHAR(24),
    detail         TEXT,                          -- JSON string
    correlation_id VARCHAR(64)
);
CREATE INDEX ix_audit_entity ON audit_event (entity_type, entity_id, occurred_at);
CREATE INDEX ix_audit_time   ON audit_event (occurred_at);

-- Transactional outbox: lets SSE replay after a dropped connection (NFR-6, Sec. 9.5)
CREATE TABLE outbox_event (
```

```
    id           BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    event_type   VARCHAR(48) NOT NULL,
    payload      TEXT        NOT NULL,
    created_at   TIMESTAMP   NOT NULL,
    published_at TIMESTAMP
);
CREATE INDEX ix_outbox_unpub ON outbox_event (id, published_at);

-- Web Push endpoints. One row per (attender, browser install). See Sec. 9.7.
CREATE TABLE push_subscription (
    id           BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    user_id      BIGINT       NOT NULL REFERENCES app_user(id),
    endpoint     VARCHAR(512) NOT NULL,   -- push service URL issued by the browser
    p256dh_key   VARCHAR(128) NOT NULL,   -- client public key (payload encryption)
    auth_secret  VARCHAR(64)  NOT NULL,   -- client auth secret
    device_label VARCHAR(64),
    created_at   TIMESTAMP    NOT NULL,
    last_ok_at   TIMESTAMP,
    fail_count   INTEGER      NOT NULL DEFAULT 0,
    revoked_at   TIMESTAMP,
    CONSTRAINT uq_push_endpoint UNIQUE (endpoint)
);
CREATE INDEX ix_push_user ON push_subscription (user_id, revoked_at);

-- Outbound push attempts, so a silent failure is visible rather than invisible.
CREATE TABLE push_delivery (
    id              BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    subscription_id BIGINT      NOT NULL REFERENCES push_subscription(id),
    outbox_event_id BIGINT      NOT NULL REFERENCES outbox_event(id),
    attempted_at    TIMESTAMP   NOT NULL,
    http_status     INTEGER,
    outcome         VARCHAR(16) NOT NULL,  -- SENT | EXPIRED | FAILED | SKIPPED
    detail          VARCHAR(255),
    CONSTRAINT uq_push_delivery UNIQUE (subscription_id, outbox_event_id)
);

-- Classifier lexicon, editable by the manager without a redeploy (Sec. 5.5)
CREATE TABLE category_term (
    id         BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    domain     VARCHAR(16)  NOT NULL,
    term       VARCHAR(64)  NOT NULL,
    weight     NUMERIC(4,2) NOT NULL DEFAULT 1.00,
    fuzzy      BOOLEAN      NOT NULL DEFAULT TRUE,
    active     BOOLEAN      NOT NULL DEFAULT TRUE,
    created_at TIMESTAMP    NOT NULL,
    CONSTRAINT uq_term UNIQUE (domain, term)
);
```

The audit row is written by an `@TransactionalEventListener(BEFORE_COMMIT)` handler, so it commits atomically with the change it describes. The outbox row is written the same way, and a poller publishes it to SSE subscribers and to Web Push afterwards — meaning a notification can never describe a transaction that rolled back, and a subscriber that reconnects can replay from its last event id.

`uq_push_delivery (subscription_id, outbox_event_id)` applies the same idea as the ledger's idempotency key (Sec. 3.6) to notifications: a retried or re-run publisher cannot buzz the same phone twice for the same event.

3.8 Portability constraints

The DDL above is restricted to the subset PostgreSQL and H2 both accept, because the offline fallback jar (Sec. 2.6) runs on H2:

Avoided	Used instead	Why
`SERIAL`	`BIGINT GENERATED BY DEFAULT AS IDENTITY`	SQL standard; both engines support it

Avoided	Used instead	Why
`JSONB`	`TEXT` holding JSON	H2's JSON type and operators differ from Postgres'; we never query inside these fields
Partial indexes (`... WHERE`)	Full indexes + `idempotency_key` uniqueness	H2 does not support partial indexes. The uniqueness guarantee we actually depend on (Sec. 3.6) is expressed as a plain `UNIQUE`, so it holds identically on both engines.
`citext`, arrays, `INTERVAL` arithmetic	`VARCHAR` + application-side normalization	Vendor-specific

Postgres-only performance indexes may be added in a separate, clearly-marked Flyway migration that the `offline` profile skips. Nothing *correctness*-bearing lives there.

3.9 Retention

Data	Retention	Mechanism
`complaint.reporter_contact`	90 days	Nightly job nulls the column; audit records the purge
`complaint.raw_text`	1 year	Nightly job replaces with `[purged]`; aggregates survive
`audit_event`, `point_ledger`	Indefinite	Required for bonus dispute resolution
`push_subscription`	Until revoked or 3 consecutive failures	Auto-pruned; endpoint is a device identifier (Sec. 11.4)
`push_delivery`	30 days	Diagnostics only
`source_row.raw_payload`	90 days	Purged; `content_hash` retained so dedupe still works

Purging `raw_payload` but keeping `content_hash` is what lets retention coexist with idempotency: we can still recognise a previously-seen row without storing what it said.

4. State Machines

4.1 Why this is formalised

Complaint status is the single most-read field in the system: it drives the attender queue, the dashboard, SLA aging, MTTR measurement and the point credit. Leaving it as an informally-updated string is how a ticket ends up `RESOLVED` with no docket, or credited twice.

We therefore define an explicit machine with a **guarded transition table**. The only way to change `complaint.status` is `WorkflowService.transition(complaintId, event, actor, payload)`, which:

1. Loads the aggregate with its `version` (optimistic lock).
2. Looks up `(currentState, event)` in the table; an absent pair throws `IllegalTransition`.
3. Evaluates the guard; failure throws `GuardFailed` with a human-readable reason.
4. Applies the state change, runs the side effects, writes the audit row and the outbox row — all in **one transaction**.

There is no other write path. Controllers, the scheduler and the CSV importer all funnel here.

4.2 Complaint lifecycle

```
stateDiagram-v2
    [*] --> NEW : ingested
    NEW --> ASSIGNED : auto_dispatch (candidate found)
    NEW --> TRIAGE : auto_dispatch (no candidate)<br/>or UNCLASSIFIED
    TRIAGE --> ASSIGNED : manager_assign
    TRIAGE --> REJECTED : manager_reject

    ASSIGNED --> ASSIGNED : manager_reassign
    ASSIGNED --> INSPECTING : attender_start
    ASSIGNED --> TRIAGE : attender_release

    INSPECTING --> AWAITING_CMS : defect_confirmed
    INSPECTING --> REJECTED : no_defect_found
    AWAITING_CMS --> IN_PROGRESS : docket_recorded
    IN_PROGRESS --> RESOLVED : mark_resolved
    IN_PROGRESS --> ON_HOLD : block (parts/access)
    ON_HOLD --> IN_PROGRESS : unblock

    RESOLVED --> CLOSED : manager_close / auto_close (7d)
    RESOLVED --> IN_PROGRESS : manager_reopen
    CLOSED --> [*]
    REJECTED --> [*]
```

Transition table

#	From	Event	To	Actor	Guard	Side effects
T1	—	`ingest`	`NEW`	System	Source row not already linked to a complaint	Create complaint, classify, set base priority + SLA due
T2	`NEW`	`auto_dispatch`	`ASSIGNED`	System	Domain ≠ `UNCLASSIFIED` **and** dispatch returns a candidate	Create task + assignment, snapshot score, emit `TaskAssigned`

#	From	Event	To	Actor	Guard	Side effects
T3	`NEW`	`auto_dispatch`	`TRIAGE`	System	Domain `UNCLASSIFIED` **or** no candidate	Record triage reason, emit `TriageRequired`
T4	`TRIAGE`	`manager_assign`	`ASSIGNED`	Manager	Target attender active; domain now set	Set domain (`classified_by = MANUAL`), create task + assignment
T5	`TRIAGE`	`manager_reject`	`REJECTED`	Manager	Reason mandatory	Close complaint, no points
T6	`ASSIGNED`	`manager_reassign`	`ASSIGNED`	Manager	New attender ≠ current; task not yet resolved	Release old assignment, create new one, audit both
T7	`ASSIGNED`	`attender_start`	`INSPECTING`	Assigned attender	Caller holds an `ACTIVE` assignment	Stamp `inspection_started_at`
T8	`ASSIGNED`	`attender_release`	`TRIAGE`	Assigned attender	Reason mandatory	Release assignment, back to triage
T9	`INSPECTING`	`defect_confirmed`	`AWAITING_CMS`	Assigned attender	—	Prompt for docket on mobile UI
T10	`INSPECTING`	`no_defect_found`	`REJECTED`	Assigned attender	Note mandatory	Close; **no points credited**
T11	`AWAITING_CMS`	`docket_recorded`	`IN_PROGRESS`	Assigned attender	Docket matches configured pattern (OQ-1) and is non-blank	Insert `cms_docket`, docket sub-state → `FILED`
T12	`IN_PROGRESS`	`block`	`ON_HOLD`	Assigned attender / Manager	Reason mandatory	Pause SLA clock
T13	`ON_HOLD`	`unblock`	`IN_PROGRESS`	Assigned attender / Manager	—	Resume SLA clock, extend due date by hold duration
T14	`IN_PROGRESS`	`mark_resolved`	`RESOLVED`	Assigned attender	Current docket exists **and** caller assignment is `ACTIVE`	**Credit points** via ledger (idempotent, Sec. 7.3); mark assignment `COMPLETED`; emit `ComplaintResolved`
T15	`RESOLVED`	`manager_close`	`CLOSED`	Manager	—	Stamp `closed_at`

#	From	Event	To	Actor	Guard	Side effects
T16	`RESOLVED`	`auto_close`	`CLOSED`	System	7 days elapsed since resolution	Stamp `closed_at`
T17	`RESOLVED`	`manager_reopen`	`IN_PROGRESS`	Manager	Reason mandatory	**Reverse the credit** (new `REVERSAL` ledger row, never a delete)

Notes on specific guards

T11 — the docket gate. `mark_resolved` (T14) requires a docket to exist. This is the mechanism that makes the proposal's institutional-traceability requirement structurally enforced rather than a matter of attender diligence: you cannot collect +2 points for a complaint you never filed with the Engineering Unit. Because there is no CMS API (Sec. 1.8), we validate *format* only, and the pattern is configuration (OQ-1) so a wrong guess costs a property change, not a redeploy.

T17 — reopen reverses rather than deletes. The ledger is append-only, so a reopen writes a compensating `REVERSAL` row. The attender's balance falls by 2, and both the original credit and the reversal remain visible in their history. Re-resolving later writes a *new* credit with a new idempotency key (`RESOLVE:{task}:{attender}:r{n}` where `n` is the reopen count), so the idempotency guarantee survives the reopen cycle without ever blocking a legitimate re-credit.

T12/T13 — the SLA clock. Hold time is excluded from SLA and from MTTR. Without this, "waiting for the Engineering Unit to send a plumber" would be recorded as attender slowness and would poison the bottleneck analysis in Sec. 8.

4.3 CMS docket sub-state

The docket has its own small lifecycle, tracked independently because it advances on the *CMS side*, outside our control, and is only reflected here when a human tells us.

```
stateDiagram-v2
    [*] --> NOT_REQUIRED : complaint rejected before inspection
    [*] --> PENDING : defect confirmed
    PENDING --> FILED : attender enters docket no.
    FILED --> SUPERSEDED : corrected entry
    SUPERSEDED --> FILED : replacement recorded
    FILED --> VERIFIED : manager confirms against CMS portal
    NOT_REQUIRED --> [*]
    VERIFIED --> [*]
```

Sub-state	Meaning	Who advances it
`NOT_REQUIRED`	Complaint closed before it ever needed filing	System
`PENDING`	Defect confirmed on site, not yet filed with the EU	System (T9)
`FILED`	A docket number has been recorded in our system	Attender (T11)
`SUPERSEDED`	An earlier docket entry was corrected; the old row is retained	Attender / Manager
`VERIFIED`	Manager has eyeballed it against the actual CMS portal	Manager

`VERIFIED` is deliberately **not** a precondition for anything. It is a bookkeeping convenience for the manager's periodic

reconciliation, because making the attender's point credit depend on a manual manager check would reintroduce exactly the daily bottleneck this project exists to remove.

4.4 Task lifecycle (special duties)

Special duties have no complaint and no docket. They are posted open and claimed.

```
stateDiagram-v2
    [*] --> OPEN : manager posts duty
    OPEN --> CLAIMED : attender claims
    OPEN --> CANCELLED : manager cancels
    OPEN --> EXPIRED : closes_at passes unclaimed
    CLAIMED --> OPEN : attender releases
    CLAIMED --> COMPLETED : attender marks done
    COMPLETED --> CLAIMED : manager reverts (reverses points)
    COMPLETED --> [*]
```

For a **shared** duty (`task.shared = true`) — the common-room overhaul case from the proposal — several attenders may hold `ACTIVE` assignments at once. The duty reaches `COMPLETED` when every active assignee has marked their part done, and **each** assignee is credited once, guarded by their own idempotency key `DUTY:{task_id}:{attender_id}`. The concurrency argument for this is in Sec. 7.5.

4.5 Priority and aging

Priority is not a fixed label; it ages so that old low-priority work eventually surfaces above new noise.

Base priority	Assigned when	Initial SLA (OQ-2)
`P1`	Safety/utility keywords hit (`shock`, `spark`, `no water`, `sewage`, `short circuit`)	Electrical 4h · Other 8h
`P2`	Default for a confidently classified complaint	Electrical 8h · RO 12h · Civil 24h · Network 24h
`P3`	Cosmetic keywords, or explicitly downgraded by the manager	72h

A scheduled job escalates `P3 → P2 → P1` when a complaint has been open past **1.5×** its SLA window without reaching `RESOLVED`, and `base_priority` is retained so the dashboard can always show "P1 (aged from P3)". Escalation is capped at `P1`; hold time (T12) does not count toward it.

5. Ingestion & Categorization

5.1 The hard part

Google Sheets is not an event source. It offers no change feed, no per-row identity and no "give me what changed since X". We poll a mutable document and must derive deltas ourselves, while guaranteeing NFR-1's headline property: **a complaint is never created twice**.

Three properties of the real sheet make the naive approach fail:

1. **Row indices are not stable.** If the manager deletes row 40, every row below shifts up by one. Keying on `(sheet_id, row_index)` would then report the entire tail of the sheet as amended and, depending on the reconciliation rule, could duplicate hundreds of complaints in one poll.
2. **Responses can be edited.** With "edit response" enabled, a resident can change their answer after submission. Content changes; it is still the same complaint.
3. **The instance sleeps.** On free-tier hosting the poller may not run for hours (Sec. 2.5). Any design that depends on "what changed since the last tick" is fragile; the design must be *stateless with respect to elapsed time*.

5.2 Column mapping

Form question wording and column order will change (OQ-4). Ingestion therefore maps by **header name**, configured, not by position:

```yaml
ingestion:
  sheets:
    columns:
      timestamp:  ["Timestamp", "Submitted at"]
      email:      ["Email Address", "Email"]
      room:       ["Room Number", "Room No", "Room"]
      description:["Describe the issue", "Complaint", "Issue"]
      category:   ["Type of issue"]        # optional resident-supplied hint
    required: [timestamp, description]
```

Each logical field lists accepted header aliases. At the start of a run we resolve headers once; an unresolvable **required** column fails the whole run loudly rather than ingesting garbage — a silent partial ingest is far worse than a visible failure, because the manager would have no way to know complaints were being dropped.

5.3 Delta detection

Identity is derived from content, not position:

```
response_key = SHA256( normalize(timestamp) || "\x1f" || normalize(email) )
content_hash = SHA256( join("\x1f", [normalize(cell) for cell in row]) )
```

`response_key` is stable across row shifts and across edits — a Google Form response keeps its submission timestamp and submitter when edited. `content_hash` changes whenever any answer changes. Splitting identity from change-detection is what makes all three failure modes above tractable.

Reconciliation rules

Observation	Meaning	Action
`response_key` unseen	New submission	Insert `source_row`, create complaint (T1)
Known key, same `content_hash`	Unchanged	Touch `last_seen_*`, nothing else
Known key, different `content_hash`	Response was edited	Insert `source_row_revision`, state → `AMENDED`, **re-classify**, raise `ComplaintAmended` for the manager. Never create a second complaint.

Observation	Meaning	Action
Known key absent from a **full** scan	Row deleted from the sheet	State → `VANISHED`. **The complaint is not deleted** — it is flagged for manager review
Duplicate `response_key` within one batch	Two submissions in the same second by the same person	Disambiguate by appending the in-batch ordinal before hashing

Marking a vanished row rather than deleting the complaint is deliberate: work may already have been done and points credited against it. Cascading a spreadsheet deletion into our operational record would corrupt the ledger.

Scan strategy

Fetching every row on every poll is wasteful at a year's volume, but a tail-only fetch cannot see edits to older rows. We therefore run two modes:

Mode	Range fetched	When	Detects
Incremental	Last `N = 300` rows	Every poll (3 min)	New submissions, recent edits
Full reconcile	Entire sheet	Nightly, and on manual "Sync Now"	Old edits, deletions (`VANISHED`)

`VANISHED` is only ever concluded from a **full** scan, never an incremental one — an incremental window naturally "misses" older rows and must not mistake that for deletion. This is the single most important guard in the algorithm.

Pseudocode

```
function reconcile(rows, run, isFullScan):
    seen = {}
    for (i, row) in rows:
        key  = responseKey(row); hash = contentHash(row)
        if key in seen: key = responseKey(row, disambiguator = ++seen[key])
        seen[key] = 0
        existing = sourceRowRepo.find(sourceRef, key)

        if existing == null:
            sr = sourceRow(key, hash, rowIndex = i, state = ACTIVE, run)
            complaint = normalizeToComplaint(row)          # 5.4
            classify(complaint)                            # 5.5
            workflow.transition(complaint, INGEST)         # T1
            run.rowsCreated++
        else if existing.contentHash != hash:
            existing.addRevision(hash, row, run)
            existing.state = AMENDED
            reclassifyAndFlag(existing.complaint)
            run.rowsAmended++
        else:
            existing.touch(run)

    if isFullScan:
        for sr in sourceRowRepo.activeNotSeenIn(run):
            sr.state = VANISHED                            # complaint survives
            run.rowsVanished++
```

Each row is processed in its **own transaction**. One malformed response therefore cannot abort the whole run; it is recorded on `ingestion_run.error_detail` and the run continues. Given that this is an unattended background job over data we do not control, partial progress is strictly better than all-or-nothing.

Quota and failure budget

Sheets API allows 300 read requests/minute/project. At one request per 3-minute poll we use ~0.1% of quota, leaving ample headroom for manual syncs. Failures back off exponentially (3 → 6 → 12 → 30 min, capped) and after three consecutive failures the dashboard shows a **degraded-ingestion banner** telling the manager to use the CSV fallback. Silence is the failure mode to avoid: the manager must find out from the UI, not by noticing complaints stopped arriving.

5.4 Normalization to a complaint

Field	Derivation
`raw_text`	The description cell, verbatim, never modified
`reported_at`	Parsed form timestamp, interpreted in Asia/Kolkata, stored UTC
`reporter_room`	Regex `\b([A-Z]{0,2}[-\s]?\d{3,4})\b` over the room cell, then the description as fallback
`floor_id`	Derived from room number via a configurable rule (default: first digit of a 3-digit room, first two of a 4-digit). Unmapped → `NULL`, which routes to triage
`reporter_contact`	Email cell; purged after 90 days (NFR-9)
`public_ref`	`CAU-{year}-{zero-padded sequence}`

5.5 Categorization

Approach (ADR-004): a weighted lexicon with fuzzy matching, gated on confidence. Apache OpenNLP document categorization is designed in as an optional second stage (Sec. 5.7) but is *not* on the critical path.

The reasoning is about data, not sophistication. A supervised classifier needs a few hundred labelled examples per class; we do not yet know we will have them, complaint text is short (often under 10 words), frequently misspelled, and often code-mixed Tamil/English transliteration. A weighted lexicon degrades gracefully in exactly those conditions, needs no training data, is editable by the manager without a redeploy, and — decisive for a viva — every decision it makes can be explained by pointing at the terms that fired.

Pipeline

```
raw text
  → lowercase, Unicode NFKC, strip punctuation, collapse whitespace
  → expand abbreviations ("ac"→"air conditioner", "wtr"→"water", "elec"→"electrical")
  → tokenize (whitespace + n-grams up to 3, so phrases can be matched)
  → match against category_term
  → score per domain
  → confidence gate
  → DOMAIN or UNCLASSIFIED
```

Matching and scoring

For each domain d, over matched terms t:

$$S(d) = \sum_{t \in T(d)} w(t) \cdot m(t)$$

where $m(t) = 1.00$ exact token/phrase match

```
            0.70   fuzzy match  (Damerau-Levenshtein ≤ 2 on tokens of length ≥ 4,
                                 or Jaro-Winkler ≥ 0.90)
            0.00   otherwise
```

Multi-word phrases carry higher weights than single tokens because they disambiguate: `water` alone is nearly useless (it appears in RO Plant, Civil and Electrical complaints), while `no water` and `water leaking` are decisive. Phrase matches suppress their constituent single-token matches so a phrase is not double-counted.

Let $S_1 \geq S_2$ be the top two domain scores. Confidence combines **absolute evidence** and **separation**:

```
confidence = 0  if S₁ < S_min                          (nothing meaningful matched)
             0  if S₁ = 0
             (S₁ − S₂) / S₁    otherwise               (margin over the runner-up)

accept if confidence ≥ τ  AND  S₁ ≥ S_min
```

Defaults: `S_min = 1.0`, τ `= 0.40`. Both are configuration, tuned against the gold set (Sec. 12.3).

Two gates rather than one matters. A single absolute threshold accepts a complaint that scores strongly for *two* domains equally — precisely the ambiguous cases where a human should decide. A single margin threshold accepts a complaint where one weak term matched and nothing else did. Requiring both routes exactly the uncertain cases to triage.

Failing to triage, on purpose

Anything not accepted becomes `UNCLASSIFIED` and lands in the manager's triage queue (T3) — the proposal's Risk 2 Plan B. The manager sets the domain in one click, and that correction is recorded. This is a deliberate precision-over-recall stance: a wrongly-auto-assigned complaint sends an attender to the wrong job and erodes trust in the whole system, whereas a triaged one costs the manager three seconds.

Priority extraction

Independent of domain, a separate keyword set raises base priority (Sec. 4.5). `shock`, `spark`, `short circuit`, `sewage`, `no water`, `overflow` → `P1`. These are matched **exactly only** — no fuzzy matching — because a false P1 from a near-miss spelling would distort the whole queue.

5.6 Meeting the 90% accuracy criterion

The acceptance criterion needs a testable definition, so we fix one now:

> Accuracy is measured over the **auto-accepted** subset (confidence $\geq \tau$) of a held-out labelled gold set, as the fraction whose predicted domain equals the human label.

Items routed to triage are excluded from the denominator but tracked separately as **coverage** (the fraction auto-accepted). Reporting both is what makes the number honest — a classifier that accepts nothing would otherwise score 100%. Targets: **accuracy ≥ 90%, coverage ≥ 70%**. Full methodology, dataset construction and the confusion matrix live in Sec. 12.3.

5.7 OpenNLP as a deferred second stage

Once enough manager triage corrections have accumulated, they form a labelled corpus for free — every triage decision is `(text, human label)`. At that point an OpenNLP `DocumentCategorizer` can be trained and consulted **only for items the lexicon routes to triage**, with its own confidence gate. This is upside without risk: the rule engine stays the primary path, so a poorly trained model can only convert triage items into accepted ones, never corrupt a decision the rules were already confident about. Tagged **Extended** in Sec. 13 — it ships only if the schedule allows.

5.8 CSV fallback

Risk 1's Plan B is a drag-and-drop `.csv` upload on the manager dashboard. It is **the same port**:

```
public interface ComplaintSource {
    String sourceRef();
    SourceType type();
    List<SourceRow> fetch(FetchWindow window);
}
```

`GoogleSheetsSource` and `CsvUploadSource` both implement it, and reconciliation, normalization, classification and dispatch are shared verbatim. The fallback therefore inherits the idempotency guarantee for free: re-uploading the same CSV twice — the obvious thing a flustered user does during an outage — produces zero duplicate complaints, because `response_key` collides.

6. Dispatch Heuristic

6.1 Problem statement

Given a classified complaint c and the set of attenders A, choose the attender who should do the work — accounting for who covers that floor, who is actually on duty, and who is already overloaded. The proposal requires this be a genuine heuristic rather than round-robin, and that it be **formally specified** so its behaviour is testable.

Requirements the formulation must satisfy:

R	Requirement
R1	Always terminates with either an assignment or an explicit, recorded triage reason
R2	Deterministic — identical inputs give identical output, including tie-breaks
R3	Explainable — the manager can see *why* an attender was chosen
R4	No starvation — one attender cannot absorb everything while others idle
R5	Tunable without a redeploy

6.2 Formulation

Stage 1 — hard eligibility filter

Availability is a **gate, not a score**. An attender on leave must never be chosen, no matter how well they score elsewhere; folding it into a weighted sum would let a strong floor match outvote the fact that someone is not at work.

```
E = { a ∈ A :  a.active
             ∧ availabilityAt(a, now) = AVAILABLE
             ∧ ¬ alreadyAssignedTo(a, c) }
```

If `E = ∅` → **no assignment**, complaint → `TRIAGE` with reason `NO_AVAILABLE_ATTENDER` (T3). This is the proposal's "all assigned floor attenders unavailable" edge case, and it is handled by construction rather than by a special branch.

Stage 2 — score the eligible set

```
Score(a, c) = w_f · Floor(a, c)
            + w_l · Load(a)
            + w_s · Skill(a, c.domain)
            + w_c · Continuity(a, c)
```

Every component is normalised to **[0, 1]** and weights sum to 1, so `Score ∈ [0, 1]` and the weights are directly interpretable as "how much does this factor matter".

Floor proximity — `Floor(a, c)` Uses the coverage in force *at the complaint's report time*, not today's (Sec. 3.4), so historical decisions stay explainable.

Situation	Value
`PRIMARY` coverage of the complaint's floor	**1.00**
`SECONDARY` coverage of that floor	**0.70**
Covers an adjacent floor in the same wing ($\lvert\Delta\rvert = 1$)	**0.40**
Same wing, $\lvert\Delta\rvert \geq 2$	`max(0.10, 0.35 − 0.05·|Δ|)`
Different wing	**0.10**

Non-zero floors for distant attenders are intentional: when the primary is unavailable, someone still has to go, and a zero would make the whole term uninformative in exactly that case.

If `c.floor_id IS NULL` (room number unparseable, OQ-4) the term is undefined. Rather than guess, the complaint goes to `TRIAGE` with reason `UNMAPPED_LOCATION` — dispatching on a location we could not read is worse than asking.

Workload — `Load(a)` Higher score = *more* free capacity, so the term pushes work toward idle attenders.

```
L(a) = Σ  π(t)          over tasks t currently ACTIVE for a
       t                (status ∈ {ASSIGNED, INSPECTING, AWAITING_CMS, IN_PROGRESS})

where π(P1) = 3,  π(P2) = 2,  π(P3) = 1        (ON_HOLD tasks count 0)

Load(a) = 1 − min(1, L(a) / L_cap)              L_cap default 8
```

Weighting by priority rather than counting tickets reflects reality: three P1 electrical faults is not the same workload as three P3 cosmetic complaints. `ON_HOLD` work is excluded because the attender is genuinely not blocked by it (Sec. 4.2, T12).

Domain skill — `Skill(a, d)` Defaults to a neutral **0.5** for every attender/domain pair, because we do not yet know whether attenders specialise (OQ-5). If specialisations exist the manager sets them; if historical data accumulates, the value can be derived from that attender's resolution rate and relative MTTR in that domain. Neutral defaults mean the term contributes nothing until we have real information — the heuristic never invents a preference it has no evidence for.

Continuity — `Continuity(a, c)`

```
1.0  if a resolved a complaint in the same room within 14 days
0.5  if a resolved a complaint on the same floor within 14 days
0.0  otherwise
```

A recurring fixture fault is faster to fix for whoever saw it last, and — usefully — it is also the same signal Sec. 8.3 uses to detect chronic defects and band-aid repairs.

Default weights

Weight	Value	Rationale
`w_f` floor	**0.45**	The stakeholder's stated primary criterion
`w_l` load	**0.30**	Strong enough to override floor when someone is badly overloaded
`w_s` skill	**0.15**	Inert until specialisations are known
`w_c` continuity	**0.10**	A tiebreak-scale nudge, not a driver

All five values, plus `L_cap`, `τ`, and the proximity table, are `@ConfigurationProperties` (R5). Sec. 12 pins their behaviour with tests that assert *relationships* ("an overloaded primary loses to an idle secondary") rather than exact float outputs, so tuning does not break the suite.

Stage 3 — select

```
best = argmax Score(a, c)
       a∈E
```

Tie-break chain, applied in order (R2, R4):

1. Fewest tasks assigned **today** — the anti-starvation term
2. Longest time since last assignment

3. Lowest `app_user.id` — guarantees full determinism

Then: create `task`, create `task_assignment` with `assign_method = AUTO`, and persist the full score breakdown into `task_assignment.score_snapshot` as JSON (R3):

```
{"chosen": 7, "score": 0.781,
 "components": {"floor": 1.0, "load": 0.625, "skill": 0.5, "continuity": 0.0},
 "weights": {"floor":0.45,"load":0.30,"skill":0.15,"continuity":0.10},
 "runnersUp": [{"attenderId": 9, "score": 0.503}],
 "tieBreak": null}
```

The snapshot is what makes "why did Ravi get this?" answerable months later, and it is what the dashboard renders in the assignment tooltip.

Capacity overflow

If every eligible attender is at or over `L_cap`, we still assign to the best-scoring one — work cannot simply vanish — but emit a `CapacityExceeded` notification so the manager sees the queue is saturated. Overflowing visibly beats triaging silently.

6.3 Shared tasks

For `task.shared = true` (common-room overhaul, furniture transport), dispatch selects the **top *k*** by score instead of the top 1, where *k* is set by the manager when posting the duty. Each selected attender gets their own `task_assignment` row and, on completion, their own ledger credit under their own idempotency key. The concurrency consequences are Sec. 7.5.

6.4 Degenerate cases

Case	Detection	Outcome
No attender available	`E = ∅`	`TRIAGE` / `NO_AVAILABLE_ATTENDER`
Room number unparseable	`floor_id IS NULL`	`TRIAGE` / `UNMAPPED_LOCATION`
Floor exists but nobody covers it	All `Floor = 0.10`	Assign anyway (least-loaded wins), warn `UNCOVERED_FLOOR`
Complaint `UNCLASSIFIED`	Domain gate	Never reaches dispatch — triaged at T3
Every attender at capacity	`∀a: L(a) ≥ L_cap`	Assign best, notify `CapacityExceeded`
Exact score tie	Equal `Score`	Tie-break chain, Sec. 6.2 stage 3
Manager disagrees	—	`manager_reassign` (T6); both assignments audited

Every row terminates (R1). There is no path where a complaint silently has no owner — the invariant the dispatch module is responsible for (Sec. 2.3).

6.5 Why greedy, and not optimal assignment

A global optimum over a batch (Hungarian algorithm, min-cost flow) is the textbook answer and is **deliberately rejected** here:

- Complaints arrive **one at a time** from the poll. A batch is typically 1–3 items, and the optimal assignment of a 1-item batch is the greedy one.
- With $|A| = 4$, the theoretical gain over a whole day is negligible, while the cost is real: batch optimisation would mean holding complaints back to accumulate a batch, adding latency to exactly the P1 cases that matter most.
- Greedy is explainable (R3). "Ravi scored 0.78, Kumar 0.50" is something the manager can argue with. "The global cost matrix minimised at this permutation" is not.

Complexity is $O(|A| \cdot \log|A|)$ per complaint — trivially within NFR-3. This trade-off is recorded as ADR-005.

7. Points Ledger & Concurrency Control

7.1 What can actually go wrong

The proposal calls for "strict transactional isolation and optimistic locking to prevent race conditions and double-crediting". Before choosing mechanisms, here is the concrete threat model — five races that will genuinely occur in this deployment:

#	Race	Realistic trigger	Consequence if unhandled
C1	**Double resolve**	Attender taps *Resolve*, the response is lost on hostel Wi-Fi, they tap again	+4 points for one job — corrupts the bonus
C2	**Lost update**	Manager reassigns a task at the same moment the attender resolves it	Credit goes to the wrong person, or the reassignment silently vanishes
C3	**Shared-task double completion**	Two attenders on a shared duty mark done simultaneously	Task completion logic runs twice; over-notification, possible double close
C4	**Over-claim**	Special duty with `k = 1` claimed by two attenders in the same second	Two people show up; one is credited for work they did not get
C5	**Torn leaderboard read**	Dashboard reads while credits are being written	Standings briefly inconsistent with the ledger

These are not hypothetical. C1 in particular is *the expected case* on a phone in a hostel corridor, not an exotic edge.

7.2 Mechanisms

We use four layered mechanisms rather than one, because each race has a different shape:

M	Mechanism	Addresses
M1	Optimistic locking (`@Version`) on every mutable aggregate	C2, C3
M2	`UNIQUE (idempotency_key)` on `point_ledger`	C1
M3	Client-supplied `Idempotency-Key` on state-changing requests	C1 (at HTTP level, before it reaches the domain)
M4	Short pessimistic lock on the task row for claims	C4
M5	Leaderboard derived by query, never stored	C5

A note on the layering: M2 alone is sufficient to make double-crediting impossible. M3 exists so the *user experience* of a retry is a clean success rather than a swallowed conflict, and M1 exists for the non-ledger state that also needs protecting.

7.3 Why double-crediting is structurally impossible

This is the central correctness argument of the system.

The credit for resolving a complaint is written with a key derived **deterministically from facts that identify the work**, not from anything about the request:

```
String key = "RESOLVE:" + task.getId() + ":" + attender.getId()
           + (reopenCount > 0 ? ":r" + reopenCount : "");
```

Crediting is then:

```
@Transactional
void credit(Task task, Attender a, int points, String reasonCode, String key) {
    try {
        ledgerRepo.saveAndFlush(new LedgerEntry(a, task, reasonCode, points, key, ...));
    } catch (DataIntegrityViolationException dup) {
        // uq_ledger_idem — this exact credit already exists. Not an error.
        log.info("Credit {} already applied; no-op", key);
    }
}
```

The guarantee does not rest on transaction ordering, isolation level, retry logic, or on the application checking-then-inserting (which is itself a race). It rests on a **database uniqueness constraint**, which the database enforces atomically no matter how many concurrent transactions attempt the insert. Whether the duplicate arrives 20 milliseconds later on a retried HTTP request or 20 minutes later from a queued offline action, the outcome is identical: one credit.

The `:r{n}` suffix handles the reopen cycle (T17). After a reopen the reopen counter increments, so a legitimate re-resolution produces a *different* key and is correctly credited again — the constraint blocks duplicates without ever blocking genuine repeat work.

Consequence: `attender.points` does not exist as a column anywhere in the schema. There is no counter to increment, therefore no lost-update on a counter, therefore nothing to reconcile. A balance is a `SUM` over immutable facts.

7.4 Idempotency at the HTTP boundary

Every state-changing attender action carries a key generated on the client:

```
POST /api/attender/tasks/42/resolve
Idempotency-Key: 6f1c2b90-...        # UUID v4, persisted in localStorage until ACKed
```

The mobile UI generates the key **before** the first send and reuses it for every retry of that same user intent. Server-side, a filter records `(key, endpoint, status, response)` for 24 hours; a replay returns the stored response without re-executing.

This is what makes the flaky-connection story work end to end. The attender taps *Resolve*, loses signal, the service worker queues the request, and when connectivity returns it replays with the same key. Whether the original request actually reached the server or not, the attender sees one success and the ledger holds one credit.

7.5 Shared tasks

For `task.shared = true`, each assignee has an independent `task_assignment` row and an independent idempotency key `DUTY:{task_id}:{attender_id}`. Two attenders completing simultaneously therefore credit *different* keys — no contention, both correct.

The contended part is the **task's own completion**, which depends on all assignees being done:

```
@Transactional
void markMyPartDone(long taskId, long attenderId, String idemKey) {
    Task task = taskRepo.findById(taskId).orElseThrow();          // loads @Version
    TaskAssignment mine = assignmentRepo.findActive(taskId, attenderId)
                          .orElseThrow(NotAssigned::new);
    mine.setStatus(COMPLETED);
    ledger.credit(task, attenderId, task.getPointValue(), "SPECIAL_DUTY", idemKey);

    if (assignmentRepo.countActiveIncomplete(taskId) == 0) {
        workflow.transition(task, TASK_COMPLETED);                // bumps task.version
    }
}
```

Under concurrency both transactions read `task.version = v`. The one that commits second finds the version has moved and throws `ObjectOptimisticLockingFailureException`. It is retried once transparently; on retry it re-reads, sees its peer

already completed the task, and skips the transition. Its own credit — already keyed uniquely — is unaffected either way. C3 resolved.

7.6 Claiming a duty (C4)

Claiming is a genuine read-modify-write on a count (`active claims < k`), and OCC handles it, but the retry-on-conflict path is user-visible and confusing for someone standing in a corridor. Since contention is trivially low — at most four people, claiming a handful of duties a week — we take a **short pessimistic lock** and give the loser an immediate, clear answer:

```
@Transactional
void claim(long taskId, long attenderId) {
    Task t = taskRepo.findByIdForUpdate(taskId);      // SELECT ... FOR UPDATE
    if (t.getStatus() != OPEN) throw new AlreadyClaimed();
    if (assignmentRepo.countActive(taskId) >= t.getSlots()) throw new NoSlotsLeft();
    assignmentRepo.save(new TaskAssignment(t, attenderId, CLAIMED));
}
```

The lock is held for microseconds. `uq_task_attender` (Sec. 3.6) additionally makes a double-claim by the *same* attender impossible regardless. Choosing pessimistic locking here, against the general preference for OCC elsewhere, is recorded as ADR-006 — the deciding factor is UX clarity at negligible contention, not performance.

7.7 Optimistic locking and conflict UX

`@Version` guards `complaint`, `task`, `task_assignment`, `floor_assignment` and `app_user`.

Situation	Handling
Background/system job hits a conflict	Retry up to 3× with jitter, then log and skip
Attender action hits a conflict	Retry once transparently; then HTTP `409` with a plain-language message
Manager action hits a conflict	HTTP `409`, UI shows the current state and asks for confirmation

C2 concretely: the manager reassigns while the attender resolves. Both mutate `task_assignment`. One commits; the other gets a 409. If the attender lost, they are told "this task was reassigned to someone else" — which is true, and far better than silently crediting a person who no longer owns the work.

Isolation level: `READ COMMITTED` (the PostgreSQL default). We deliberately do not raise it. Every invariant we depend on is enforced by a uniqueness constraint or a version check, both of which hold at `READ COMMITTED`. `SERIALIZABLE` would add serialization failures and retry complexity to buy a guarantee we have already obtained more cheaply. Recorded as ADR-007.

7.8 The leaderboard

A derived projection, never stored state:

```
SELECT u.id, u.display_name,
       COALESCE(SUM(l.points), 0)                                        AS total,
       COALESCE(SUM(CASE WHEN l.points > 0 THEN l.points END), 0)        AS earned,
       COALESCE(SUM(CASE WHEN l.points < 0 THEN l.points END), 0)        AS deducted,
       COUNT(CASE WHEN l.reason_code = 'COMPLAINT_RESOLVED' THEN 1 END)  AS resolved
  FROM app_user u
  LEFT JOIN point_ledger l ON l.attender_id = u.id AND l.period_key = :period
 WHERE u.role = 'ATTENDER' AND u.active
 GROUP BY u.id, u.display_name
 ORDER BY total DESC, resolved DESC, u.display_name;
```

`LEFT JOIN` so an attender with zero activity still appears — a missing row would read as a bug to the manager. Ordering is fully deterministic through the tie-breaks.

At NFR-1 scale (~50k ledger rows/year, ~600/month per period) this is an index scan over a few hundred rows and needs **no caching, no materialized view and no denormalised counter**. Adding one would trade away the very property that makes the ledger correct. If volume ever justified it, the right move would be a materialized view refreshed on commit — not a mutable counter.

7.9 Month boundaries

`period_key` is derived from `effective_date`, which is the **Asia/Kolkata business date** (NFR-7), not the UTC instant. A task resolved at 02:00 IST on 1 October belongs to October, which is what the manager expects; storing UTC and grouping naively would have silently filed it under September.

A penalty for late September behaviour, logged on 2 October, is entered with an explicit `effective_date` of the September date. It lands in the September period, the audit records both the effective date and the entry date, and if the September leaderboard has already been exported the manager is warned that a closed period is being amended. We allow it — the alternative is a manager who cannot correct a genuine mistake — but never silently.

8. Analytics & Telemetry

8.1 Where the numbers come from

Analytics reads only from `audit_event` and `point_ledger`. Because every status change is written through the single transition path (Sec. 4.1) with `from_state`, `to_state` and `occurred_at`, the lifecycle of any complaint is fully reconstructible from the log — we never needed to add special-purpose measurement columns, and no metric can drift out of sync with the operational data.

Stage durations are the intervals between consecutive transitions:

Stage	From → To	What it actually measures
Dispatch latency	`NEW` → `ASSIGNED`	Our system's own responsiveness
Pickup latency	`ASSIGNED` → `INSPECTING`	How long before an attender starts
Inspection latency	`INSPECTING` → `AWAITING_CMS`	Time to confirm the defect on site
Filing latency	`AWAITING_CMS` → `IN_PROGRESS`	Delay in getting a CMS docket
Repair latency	`IN_PROGRESS` → `RESOLVED`	The actual fix, excluding holds
Closure latency	`RESOLVED` → `CLOSED`	Manager confirmation lag

Time spent in `ON_HOLD` is subtracted from the enclosing stage (Sec. 4.2, T12). Without that, "waiting three days for the Engineering Unit to send a plumber" would be recorded as attender slowness and would point the bottleneck analysis at exactly the wrong place.

8.2 MTTR and bottleneck detection

Resolution time = `RESOLVED.occurred_at − complaint.reported_at − holdDuration`.

The proposal asks for MTTR, which conventionally means the arithmetic mean. We compute and display the mean, but present the **median as the headline figure**, with p90 alongside:

At ~20 complaints/day, one complaint that sat over a long weekend moves the mean by hours while the typical experience is unchanged. Reporting only the mean would make the dashboard react to noise and would quietly mislead the manager about what is normal. Showing mean, median and p90 together costs nothing and makes the skew visible rather than hidden.

Bottleneck identification, per domain d:

$$\text{For each stage } s\text{: } \quad \text{share}(s, d) = \frac{\text{median duration of } s \text{ in } d}{\Sigma \text{ median duration of all stages in } d}$$

$$\text{bottleneck}(d) = \operatorname{argmax} \text{share}(s, d), \quad \text{reported only when share} \geq 0.35$$

The 0.35 floor exists so we do not report a "bottleneck" when time is spread evenly — a stage that is 26% of a five-stage pipeline is not a bottleneck, it is arithmetic. Each result is displayed with its sample size, and suppressed entirely below `n = 10` for that domain.

SLA targets driving breach metrics (OQ-2, pending confirmation):

Domain	P1	P2	P3
Electrical	4h	8h	72h
RO Plant	6h	12h	72h
Civil	8h	24h	72h
Network	8h	24h	72h

Measured in working hours (08:00–20:00 IST, OQ-6). These are configuration, not constants.

8.3 Anomaly detection

The honest constraint

This deserves stating plainly, in the design doc rather than after the fact: with **4 attenders and ~20 complaints/day**, we are doing statistics on small samples. A textbook z-score over a 30-day window has $n \approx 600$ overall but only $n \approx 150$ per domain and $n \approx 5$ per domain-day. Ordinary mean-and-standard-deviation outlier detection on data this thin and this skewed will produce frequent false alarms, and a dashboard that cries wolf is one the manager stops reading — which is a worse outcome than having no anomaly detection at all.

Three design responses follow from this:

1. **Robust statistics only.** Median and MAD, never mean and σ, for outlier detection. Both have a 50% breakdown point; the mean has 0%.
2. **Minimum sample gates.** Every detector declares an n below which it emits nothing.
3. **Evidence with every alert.** Each alert carries its statistic, its threshold and its n, so a sceptical manager can judge it rather than trust it.

Detectors

A1 — Prolonged resolution latency. Modified z-score (Iglewicz & Hoaglin):

```
MAD  = median( |xᵢ − median(x)| )
Mᵢ   = 0.6745 · (xᵢ − median(x)) / MAD
flag if Mᵢ > 3.5
```

Reference set: same domain, trailing 90 days, resolved complaints. **Gate: n ≥ 12.** If `MAD = 0` (possible when most complaints resolve in an identical rounded time) fall back to flagging `xᵢ > 3 × median`, since the modified z-score is undefined there.

A2 — Chronic location / band-aid repair. No statistics at all, just counting:

```
flag room r if  ≥3 complaints in the same domain from r within 30 days
severity HIGH if ≥3 within 14 days
```

Exact counting is *more* reliable than a statistical test at this sample size, and this detector directly serves the proposal's "chronic fixture defects / band-aid repairs" requirement. It is also the most operationally actionable output the system produces: three electrical faults in room 214 in a fortnight means the fixture needs replacing, not another patch.

A3 — Arrival volume surge. Poisson tail test against an EWMA baseline:

```
L_t = α·x_t−₁ + (1−α)·L_t−₁          α = 0.3
flag if  P(X ≥ x_t | Poisson(L_t)) < 0.01
```

Poisson rather than Gaussian because daily arrival counts are small non-negative integers — exactly what Poisson models and where a normal approximation is worst. **Gate: ≥14 days of history.** Separate baselines per wing, so a burst localised to one wing (a failed riser, a water outage) is caught even when the hostel total looks normal.

A4 — SLA breach cluster. Flag when breaches in a rolling 7 days exceed 2× the trailing 8-week rate for that domain. **Gate: ≥4 weeks of history.**

A5 — Workload imbalance. Flag when `max(L(a)) / mean(L(a)) > 2.0` across active attenders for 3 consecutive days — this is a check on our own dispatch heuristic (Sec. 6) as much as on the attenders.

Alert storage

```sql
CREATE TABLE analytics_alert (
    id           BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    detector     VARCHAR(8)   NOT NULL,      -- A1..A5
    severity     VARCHAR(8)   NOT NULL,      -- INFO | WARN | HIGH
    subject_type VARCHAR(24)  NOT NULL,      -- COMPLAINT | ROOM | DOMAIN | ATTENDER | WING
    subject_ref  VARCHAR(64)  NOT NULL,
    summary      VARCHAR(255) NOT NULL,
```

```
    evidence      TEXT         NOT NULL,      -- JSON: statistic, threshold, n, window
    detected_at   TIMESTAMP    NOT NULL,
    acknowledged_at TIMESTAMP,
    acknowledged_by BIGINT REFERENCES app_user(id),
    dedupe_key    VARCHAR(128) NOT NULL,
    CONSTRAINT uq_alert_dedupe UNIQUE (dedupe_key)
);
```

`dedupe_key` (e.g. `A2:ROOM:214:ELECTRICAL:2026-09`) means a nightly scan re-raising the same finding updates rather than duplicates it — otherwise a genuine month-long chronic-defect condition would generate thirty identical alerts and bury everything else.

Detectors run on a nightly `@Scheduled` scan, holding the same ShedLock as ingestion (Sec. 2.5). They are pure reads over `audit_event`; they never write to operational tables.

8.4 Dashboard visualizations

Chart.js, all fed from JSON endpoints (Sec. 9.4):

Visualization	Type	Answers
Arrival velocity	Line, 30 days, per domain	"Are complaints trending up?"
Resolution funnel	Stacked bar by stage	"Where does time go?"
MTTR by domain	Bar + median/p90 error markers	"Which category is slowest?"
Attender workload	Horizontal bar, live	"Who is swamped right now?"
SLA breach distribution	Donut by domain	"What are we failing?"
Leaderboard	Ranked table, live via SSE	"Who is ahead this month?"
Anomaly feed	Card list, severity-sorted	"What needs my attention?"

Every chart that renders a derived statistic shows its **sample size**. A chart drawn from six data points must not look as authoritative as one drawn from six hundred, and the *n* is the cheapest possible way to convey that.

8.5 Explicit limitations

Recorded here so they can be stated honestly in the viva rather than discovered by a grader:

- Anomaly detection is **advisory**, never automated action. Nothing is reassigned or escalated on the strength of a detector firing.
- A1 assumes resolution times within a domain are drawn from one distribution. A domain with two genuinely distinct populations (5-minute fuse swaps and 3-day rewiring jobs) will flag the long tail as anomalous. Mitigation: segment by priority once volume allows.
- A3's baseline is contaminated by the surges it is meant to detect — a sustained high period becomes the new normal within roughly $1/\alpha \approx 3$ days.
- The first four weeks after go-live produce almost no alerts by design, because every gate is unmet. This is correct behaviour and should not be read as a defect.

9. API & Data Contracts

9.1 Shape of the web layer

A hybrid, chosen to match the two very different clients:

- **Thymeleaf server-rendered pages** deliver the page shell, navigation and first paint. For the attender on a low-end phone this matters — the queue is usable before any JavaScript runs, which is the difference between a working tool and a spinner in a corridor with two bars of signal.
- **JSON endpoints under `/api/**`** serve dynamic data: chart series, live queue refreshes, and every state-changing action.
- **SSE at `/api/manager/stream`** pushes live updates to the manager dashboard.
- **Web Push** delivers assignment alerts to attender phones even when the app is closed (Sec. 9.6).

All JSON endpoints are versionless (`/api/...`) — this is a single-deployment internal tool with no third-party consumers, so a version prefix would be ceremony without a payer.

9.2 Conventions

Aspect	Rule
Timestamps	ISO-8601 UTC with `Z` (`2026-09-08T14:32:00Z`). Clients render in Asia/Kolkata
Dates	`YYYY-MM-DD`, always the Asia/Kolkata business date
Money/points	Integers. No floats anywhere in the ledger
Nulls	Omitted from responses rather than sent as `null`
Casing	`camelCase`
Paging	`?page=0&size=25`, envelope `{content, page, size, totalElements, totalPages}`
Auth	Session cookie (`HttpOnly; Secure; SameSite=Lax`) + device token for attenders
CSRF	Spring Security tokens on all non-GET form posts
Idempotency	`Idempotency-Key` header **required** on attender state-changing POSTs (Sec. 7.4)

9.3 Attender API (mobile)

Method	Path	Purpose	Transition
`GET`	`/api/attender/me`	Profile, month points, rank	—
`GET`	`/api/attender/queue`	Assigned active tasks, priority-ordered	—
`POST`	`/api/attender/tasks/{id}/start`	Begin inspection	T7
`POST`	`/api/attender/tasks/{id}/confirm-defect`	Defect is real	T9
`POST`	`/api/attender/tasks/{id}/no-defect`	Nothing wrong found	T10
`POST`	`/api/attender/tasks/{id}/docket`	Record CMS docket number	T11
`POST`	`/api/attender/tasks/{id}/resolve`	Mark fixed → **+2 points**	T14
`POST`	`/api/attender/tasks/{id}/hold`	Block with reason	T12
`POST`	`/api/attender/tasks/{id}/release`	Hand back to triage	T8
`GET`	`/api/attender/duties`	Open special duties	—
`POST`	`/api/attender/duties/{id}/claim`	Claim a slot	Sec. 7.6
`POST`	`/api/attender/duties/{id}/complete`	Mark my part done	Sec. 7.5

Method	Path	Purpose	Transition
`GET`	`/api/attender/standing`	Point history, this period	—
`GET`	`/api/attender/push/key`	VAPID public key for subscription	—
`POST`	`/api/attender/push/subscribe`	Register this device for push	—
`POST`	`/api/attender/push/unsubscribe`	Stop push on this device	—
`POST`	`/api/attender/push/test`	Send a test notification to self	—

`GET /api/attender/queue`

```
{
  "attenderId": 7,
  "generatedAt": "2026-09-08T09:14:02Z",
  "tasks": [
    {
      "taskId": 412,
      "complaintRef": "CAU-2026-0417",
      "priority": "P1",
      "agedFrom": "P2",
      "domain": "ELECTRICAL",
      "title": "Fan not working, sparking sound",
      "room": "214",
      "floor": "Wing A, Floor 2",
      "status": "ASSIGNED",
      "reportedAt": "2026-09-08T08:41:00Z",
      "slaDueAt": "2026-09-08T12:41:00Z",
      "slaState": "AT_RISK",
      "docket": null,
      "nextActions": ["start", "release"],
      "points": 2
    }
  ],
  "counts": {"assigned": 3, "inProgress": 1, "onHold": 0}
}
```

`nextActions` is computed **server-side from the state machine**, not inferred by the client. The mobile UI renders exactly the buttons this array names. This keeps Sec. 4's transition table as the single source of truth for what is legal — the client cannot offer an action the server would reject, so an attender never taps a button that fails.

`POST /api/attender/tasks/{id}/resolve`

```
POST /api/attender/tasks/412/resolve
Idempotency-Key: 6f1c2b90-8e44-4c1a-9f2b-3d5e7a1c0b22
Content-Type: application/json

{"note": "Replaced fan capacitor and switch"}
```

```
{
  "taskId": 412,
  "status": "RESOLVED",
  "resolvedAt": "2026-09-08T11:02:41Z",
  "pointsAwarded": 2,
  "periodTotal": 34,
  "alreadyApplied": false
}
```

`alreadyApplied: true` on a replay — the request succeeded, no second credit was written (Sec. 7.3). The client shows the same success screen either way; the field exists for diagnostics, not for the user.

9.4 Manager API

Method	Path	Purpose
GET	/api/manager/summary	Dashboard header counts
GET	/api/manager/complaints	Filter/paginate all complaints
GET	/api/manager/triage	Unclassified + unassignable queue
POST	/api/manager/complaints/{id}/classify	Set domain manually (T4)
POST	/api/manager/complaints/{id}/assign	Assign from triage (T4)
POST	/api/manager/complaints/{id}/reassign	Move to another attender (T6)
POST	/api/manager/complaints/{id}/reject	Reject with reason (T5)
POST	/api/manager/complaints/{id}/reopen	Reopen, reverses credit (T17)
POST	/api/manager/complaints/{id}/act-as	**Assisted admin log** (Risk 3 Plan B)
POST	/api/manager/penalties	Log a −1 infraction
POST	/api/manager/duties	Post a special duty
POST	/api/manager/adjustments	Manual point adjustment, note required
GET	/api/manager/leaderboard?period=2026-09	Standings
GET	/api/manager/analytics/mttr	MTTR + median + p90 by domain
GET	/api/manager/analytics/arrivals	Arrival velocity series
GET	/api/manager/analytics/funnel	Stage-share breakdown
GET	/api/manager/analytics/workload	Live per-attender load
GET	/api/manager/alerts	Anomaly feed
POST	/api/manager/alerts/{id}/ack	Acknowledge an alert
POST	/api/manager/ingestion/sync	**Sync Now** (Sec. 2.5)
POST	/api/manager/ingestion/csv	CSV fallback upload (Risk 1 Plan B)
GET	/api/manager/reports/daily.xlsx	Daily Excel export
GET	/api/manager/reports/weekly.xlsx	Weekly Excel export
GET/POST	/api/manager/users	List / provision accounts
POST	/api/manager/users/{id}/reset-pin	Issue a new PIN
POST	/api/manager/users/{id}/revoke-devices	Lost-phone response
POST	/api/manager/availability	Mark an attender on leave

`act-as` is worth noting: it lets the manager perform an attender's transition during roll-call when that attender cannot use their phone. It executes the identical transition through the identical state machine, but writes `actor_id = manager, on_behalf_of = attender` to the audit log. The points still go to the attender — the manager is the typist, not the earner — and the audit record makes that distinction permanent.

`POST /api/manager/penalties`

```
{"attenderId": 9, "points": -1, "reasonCode": "LATE_ARRIVAL",
 "effectiveDate": "2026-09-08", "note": "Reported 45 min late, no prior intimation"}
```

`note` is mandatory for every negative adjustment. A point deduction affects someone's bonus; an unexplained one is indefensible in a dispute, which is precisely when the record gets read.

9.5 SSE contract

```
GET /api/manager/stream
Accept: text/event-stream
Last-Event-ID: 88213

id: 88214
```

```
event: ComplaintCreated
data: {"complaintId":903,"ref":"CAU-2026-0418","domain":"CIVIL","priority":"P2","room":"310"}

id: 88215
event: TaskResolved
data: {"taskId":412,"attenderId":7,"attenderName":"Ravi","points":2,"periodTotal":34}
```

Event	Fired when
`ComplaintCreated`	Ingestion creates a complaint (T1)
`ComplaintAmended`	Source row edited (Sec. 5.3)
`TriageRequired`	Complaint needs manager attention (T3)
`TaskAssigned`	Dispatch assigns (T2)
`TaskStarted` / `TaskResolved`	Attender progress (T7 / T14)
`DocketRecorded`	CMS number entered (T11)
`DutyClaimed` / `DutyCompleted`	Special duty progress
`PenaltyLogged`	Manager logs an infraction
`AlertRaised`	Anomaly detector fires
`IngestionDegraded`	Poll failing repeatedly (Sec. 5.3)
`PushDeliveryFailed`	An attender's device stopped accepting push (Sec. 9.6)

Event `id` is `outbox_event.id`. On reconnect the browser automatically sends `Last-Event-ID` and the server replays everything after it from the outbox. This is what makes the free-tier proxy dropping an idle connection (NFR-6) a non-event rather than a silent gap in the manager's view. Heartbeat comment every 20 s keeps the connection from being reaped in the first place. Replay is capped at 500 events; beyond that the client is told to refresh outright.

9.6 Push notification contract

Attenders receive alerts on their phone even when the app is closed, using the **Web Push protocol with VAPID** (ADR-011). The same service worker that backs the offline queue (Sec. 10.2) receives them, so no second client-side runtime is introduced.

Subscription flow

```
1. Attender logs in on their phone, taps "Enable alerts"
2. Browser asks for notification permission       (one-time, OS-level)
3. Client GETs /api/attender/push/key             → VAPID public key
4. Client calls pushManager.subscribe(...)        → endpoint + p256dh + auth
5. Client POSTs those to /api/attender/push/subscribe
6. Server stores a push_subscription row keyed on the endpoint
```

```
POST /api/attender/push/subscribe
Content-Type: application/json

{
  "endpoint": "https://fcm.googleapis.com/fcm/send/dQw4w9...",
  "keys": {"p256dh": "BNcRdreALRFXTkOOUHK1EtK2wtaz5Ry4YfYCA_0QTpQtUbVlUls0VJXg7A8u-Ts1XbjhazAkj7I99e8QcYP7DkM=",
           "auth":   "tBHItJI5svbpez7KI4CCXg=="},
  "deviceLabel": "Ravi's phone"
}
```

`endpoint` is unique (Sec. 3.7), so re-subscribing the same browser updates rather than duplicates the row — a browser reissues its endpoint periodically, and without that constraint an attender would accumulate stale subscriptions and receive the same alert several times.

Notification types

Only events an attender must **act on** are pushed. Everything else stays in the app.

Trigger	Title	Body	Priority
`TaskAssigned` (P1)	! Urgent: Room 214	Electrical — sparking sound. Due in 4h	high
`TaskAssigned` (P2/P3)	New task: Room 118	Civil — door latch broken	normal
Priority escalated to P1	T Now urgent: Room 305	RO Plant — overdue	high
`SpecialDutyPosted`	New duty available (+5)	Hostel night preparation	normal
`TaskReassigned` away	Task reassigned	Room 118 moved to Kumar	normal
SLA breach imminent (30 min)	T Due soon: Room 214	Not yet resolved	high

Deliberately not pushed: point credits, leaderboard changes, manager dashboard activity, anomaly alerts. Notifying someone that they earned the points they just earned is noise, and an attender who learns to swipe our notifications away will miss the P1 that matters.

Payload

Encrypted per the Web Push spec (`aes128gcm`); the push service never sees the plaintext.

```
{
  "title": "! Urgent: Room 214",
  "body": "Electrical — sparking sound. Due in 4h",
  "tag": "task-412",
  "data": {"taskId": 412, "url": "/m/tasks/412"},
  "actions": [{"action": "open", "title": "Open"}],
  "renotify": false
}
```

`tag` collapses repeat notifications for the same task, so an escalation replaces the original alert rather than stacking on it.

Delivery semantics

Push is **best-effort and advisory** (NFR-10). The in-app queue remains the source of truth — an attender who never enables notifications loses nothing but convenience, and no state transition depends on a push being delivered. Push sends therefore happen **after commit**, from the outbox poller, never inside the transaction that caused them.

Response from push service	Meaning	Action
`201` / `200`	Accepted for delivery	Record `SENT`
`404` / `410`	Subscription expired	Revoke the row permanently
`429`	Rate limited	Back off, retry per `Retry-After`
`413`	Payload too large	Truncate body, retry once
`5xx`	Push service down	Retry ×3 with backoff, then `FAILED`

Three consecutive failures revoke the subscription and surface a “notifications not reaching this device” note on the manager’s attender-admin screen — a phone that silently stopped receiving alerts is exactly the failure the manager needs told about, not one to discover during a P1.

9.7 Error model

RFC 7807 problem details:

```
{
  "type": "https://hostel.iitm/errors/illegal-transition",
  "title": "Action not allowed in current state",
  "status": 409,
  "detail": "Task 412 is RESOLVED; 'start' is only valid from ASSIGNED.",
```

```
  "instance": "/api/attender/tasks/412/start",
  "code": "ILLEGAL_TRANSITION",
  "currentState": "RESOLVED"
}
```

code	HTTP	Meaning
ILLEGAL_TRANSITION	409	Event not legal from the current state (Sec. 4.2)
GUARD_FAILED	422	Legal transition, unmet precondition (e.g. resolve with no docket)
OPTIMISTIC_LOCK	409	Concurrent modification; client should refresh
ALREADY_CLAIMED	409	Duty taken (Sec. 7.6)
NO_SLOTS_LEFT	409	Shared duty full
NOT_ASSIGNED	403	Caller does not hold an active assignment
INVALID_DOCKET	422	Docket fails the configured pattern (OQ-1)
INGESTION_COLUMN_MISSING	422	Required sheet column unresolvable (Sec. 5.2)
ACCOUNT_LOCKED	423	Too many failed PIN attempts (Sec. 11.2)
VALIDATION_FAILED	400	Bean-validation failure; `errors[]` lists fields

`detail` is written to be shown directly to the user. On the attender's phone there is no developer to interpret a stack trace, and "This task was already resolved" is the entire message that person needs.

10. Frontend Design

10.1 Two clients, two very different users

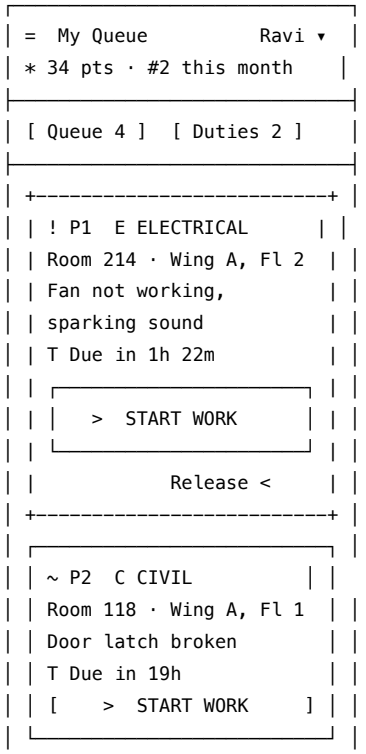
	Manager (desktop)	Attender (mobile)
Device	Office desktop, 1440px+	Low-end Android, 360px
Session	Long, seated, focused	30 seconds, standing, one hand
Network	Office LAN	Patchy hostel Wi-Fi / mobile data
Literacy	Comfortable with spreadsheets	May have limited English literacy
Needs	Density, overview, control	One task, one obvious action

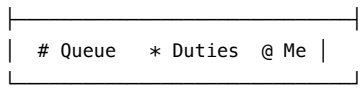
These are not the same application with a different breakpoint. They are two layouts sharing a backend, and the attender view is designed **mobile-only** — it is never used on a desktop, so it carries no desktop compromise.

10.2 Attender mobile

Design rules, all driven by the table above:

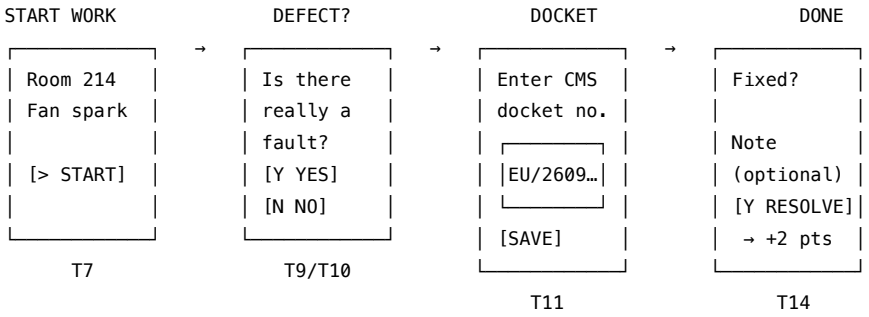
1. **Never more than one primary action visible per task.** The state machine already knows what comes next (`nextActions`, Sec. 9.3); the UI renders that one button large.
2. **Tap targets ≥ 48×48 px**, single-column, no horizontal scrolling.
3. **Icon + colour carry meaning, text confirms it.** Domain and priority are recognisable without reading.
4. **Minimal typing.** The only free-text entries in the whole flow are the CMS docket number and an optional resolution note.
5. **Optimistic UI with an offline queue.** Actions apply locally, queue when offline, replay with their idempotency key (Sec. 7.4).
6. **Push alerts for new work.** The attender should not have to poll the app to find out a P1 was assigned to them (Sec. 9.6). Permission is requested *after* the first successful login, never on a cold first visit — a permission prompt before any value is shown is the reliable way to get it denied forever.





The resolution flow

Four screens, each one decision:



The docket screen cannot be skipped — it is the guard on T14 (Sec. 4.2), and putting it inline in the flow rather than as a separate chore is what makes institutional traceability happen without nagging.

Offline behaviour

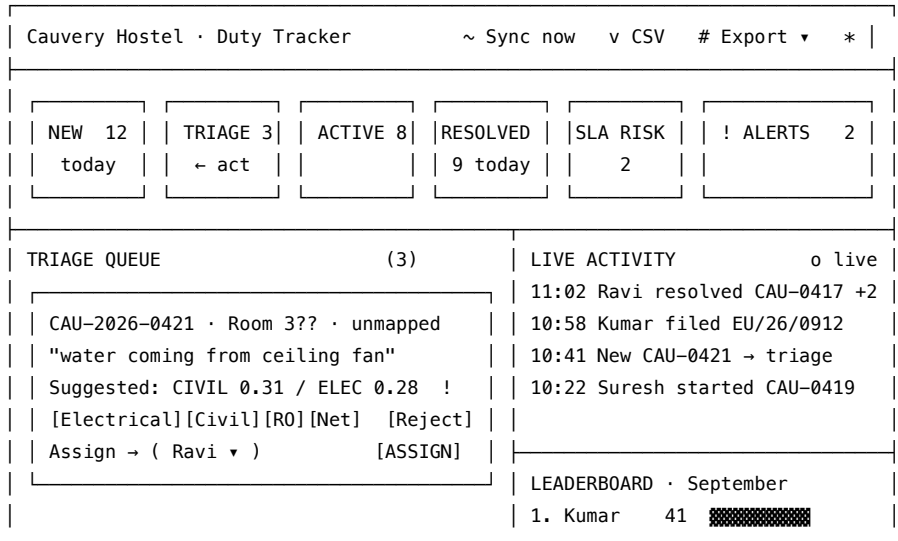
A service worker caches the shell and the current queue. Actions taken offline show a **"~ will sync"** badge, are stored in IndexedDB with their idempotency key, and replay on reconnect. Because replay is idempotent (Sec. 7.3), the failure mode "attender resolved three tasks in the basement with no signal" resolves itself correctly and silently.

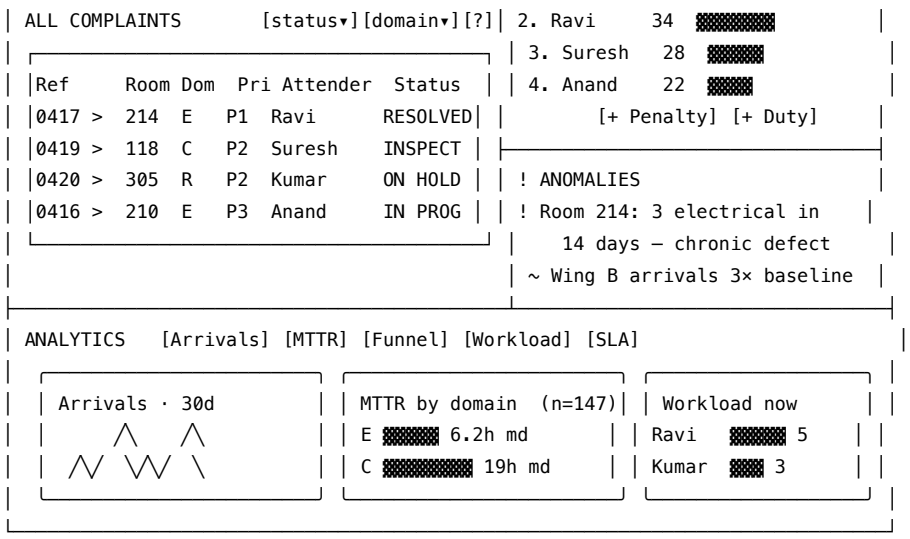
The **same service worker** receives Web Push (Sec. 9.6), which is why the app is a PWA at all: one registration serves both offline replay and notifications. Tapping a notification deep-links straight to `/m/tasks/{id}`, so the path from buzz to action is a single tap.

Notification settings

`/m/me` carries a simple toggle set — all alerts, urgent (P1) only, or off — plus a "send test notification" button. Attenders who find alerts intrusive can dial them down rather than revoking browser permission entirely, which would be irreversible from inside the app.

10.3 Manager desktop





Design notes

Triage is placed above the complaint list, not in a tab. It is the only part of the dashboard that *requires* the manager's action, and the entire value proposition is reducing their daily 30–45 minutes. Anything requiring action goes where the eye lands first.

The triage card shows the classifier's runner-up scores (`CIVIL 0.31 / ELEC 0.28`). This turns an opaque failure into useful information: the manager sees *why* the system was unsure, which makes the one-click correction feel like collaboration rather than cleanup after a dumb machine.

Live activity and the leaderboard are SSE-driven. No polling, no refresh button — the proposal's "real-time duty updates" requirement.

Every derived statistic shows its `n` (Sec. 8.4).

10.4 Screens

Screen	Route	Audience
Login	`/login`	Both — role-appropriate form (password vs PIN keypad)
Manager dashboard	`/`	Manager
Complaint detail	`/complaints/{ref}`	Manager — full audit timeline, score breakdown, docket history
Attender queue	`/m`	Attender
Task detail / flow	`/m/tasks/{id}`	Attender
Special duties	`/m/duties`	Attender
My standing	`/m/me`	Attender — point history, month rank, notification settings
Attender admin	`/admin/attenders`	Manager — provisioning, PIN reset, leave, floor coverage
Reports	`/reports`	Manager — date range, export
Settings	`/settings`	Manager — lexicon, weights, thresholds, SLA table

The **complaint detail** screen is the explainability surface: the full audit timeline, the dispatch score snapshot (Sec. 6.2) rendered as a readable breakdown, the docket history including superseded entries, and every ledger movement attached to the task. If the manager ever asks "why did this happen", one screen answers it.

10.5 Styling & accessibility

Tailwind CSS (utility classes, no separate design system to maintain solo). Chart.js for visualizations.

- WCAG AA contrast throughout; **colour is never the only signal** — priority carries an icon and a text label as well as a hue, which also covers colour-blind users
- Semantic HTML, labelled form controls, visible focus rings
- Mobile: no fixed-position overlays that a soft keyboard can trap
- Charts have accessible data-table fallbacks

10.6 Performance

Target	Approach
Attender first load < 150 KB (NFR-4)	Server-rendered shell, Tailwind purged, no framework runtime, Chart.js loaded **only** on manager routes
p95 < 500 ms (NFR-3)	Indexed queries (Sec. 3), no N+1 (fetch joins on queue and dashboard), pagination everywhere
Chart payloads	Pre-aggregated server-side; the browser never receives raw rows

11. Security & Privacy

11.1 Threat model

This is an internal tool for one hostel with five accounts, deployed on public cloud hosting. Naming what we are and are not defending against keeps the controls proportionate:

Threat	In scope	Response
Opportunistic internet scanning of a public URL	[yes]	Auth on everything, no anonymous endpoints
A student discovering the URL and trying to log in	[yes]	Rate limiting, lockout, no self-registration
An attender viewing or altering another attender's points	[yes]	Role + ownership checks on every action
An attender gaming the point system	[yes]	Docket gate on credit, append-only ledger, audit
A lost or shared attender phone	[yes]	Device-token revocation, PIN reset, push subscription revocation
Resident PII exposure	[yes]	Minimisation, retention limits (Sec. 11.4)
Targeted attack by a skilled adversary	[no]	Out of scope; no assets justify it
DDoS	[no]	Free-tier platform's problem

The most *likely* real incident here is mundane: a shared or lost phone. The controls above are weighted accordingly.

11.2 Authentication

Per ADR-002: **local credentials only, invite-only provisioning, no OAuth.**

	Manager	Attender
Credential	Username + password (min 12 chars)	6-digit PIN, unique per user
Hash	BCrypt, cost 12	BCrypt, cost 12
Session	Standard, 12h idle timeout	Standard + 90-day device token
Re-auth	Every session	Once per device

Each attender has their own PIN. PINs are never shared and never stored in plaintext: on provisioning the system generates a random 6-digit value, displays it **once** to the manager to hand over, and persists only the BCrypt hash. Neither the manager nor the database can recover it afterwards — a forgotten PIN is reset, not looked up.

A 6-digit PIN is weak on its own (10^6 space) and the design says so explicitly rather than implying otherwise. It is acceptable here only because of the surrounding controls:

- Lockout after **5** failed attempts, 15-minute cooldown, escalating on repeat
- Global rate limit of 10 login attempts/minute/IP
- Attender identity is selected from a 4-name list, so there is no username to enumerate
- Every attempt, successful or not, is written to `audit_event`
- Manager is notified in-app on any lockout
- An attender's blast radius is small: they can see their own queue and their own points, and can neither view nor alter anyone else's

Sustained brute force would need ~10^5 attempts against a 5-per-15-minutes lockout — decades. The realistic attack is a **shoulder-surfed or shared PIN**, which lockouts cannot prevent; that is why the manager can revoke devices and rotate PINs at will, and why the audit log records who did what.

Device tokens

Split-token scheme (Sec. 3.3): a public `selector` for indexed lookup and a `validator` stored only as SHA-256. Cookie is `HttpOnly; Secure; SameSite=Lax`, 90-day expiry, **rotated on every use** — so a stolen cookie that is used after the legitimate holder next authenticates fails, and the mismatch is itself a detectable signal. Manager can revoke all of one user's devices in one click.

11.3 Authorization

Role-based, enforced at the service layer (not only in controllers) via `@PreAuthorize`:

Capability	Manager	Attender
View all complaints	[yes]	[no] (own assignments only)
Trigger ingestion / upload CSV	[yes]	[no]
Classify, assign, reassign, reject, reopen	[yes]	[no]
Start / inspect / docket / resolve	[yes] (via `act-as`)	[yes] (own assignments only)
Claim special duties	[no]	[yes]
Log penalties, adjustments, special duties	[yes]	[no]
View leaderboard	[yes] (full)	[yes] (ranks + own detail)
View another attender's point detail	[yes]	[no]
Provision users, reset PINs, revoke devices	[yes]	[no]
Export reports	[yes]	[no]
Edit lexicon, weights, SLA table	[yes]	[no]

Ownership is checked in addition to role: an attender acting on a task must hold an `ACTIVE task_assignment` for it (`NOT_ASSIGNED` → 403). Role alone is never sufficient for a task action — otherwise any attender could resolve any other attender's work and collect the points.

11.4 PII and data minimisation

Data	Why we hold it	Retention (NFR-9)
Resident email	Contacting the reporter if details are unclear	90 days, then nulled
Room number	**Operationally essential** — dispatch depends on it	1 year
Complaint text	Diagnosis, classification, chronic-defect detection	1 year, then `[purged]`
Attender name	Assignment, leaderboard	Life of employment
Attender PIN	Authentication	Hash only, never plaintext
Push endpoint + keys	Delivering assignment alerts	Until revoked or 3 failures; a device identifier, so treated as personal data

We **do not** collect resident names or phone numbers even when the Form offers them — the mapped column set (Sec. 5.2) simply omits them. The cheapest way to protect data is not to hold it.

Aggregates survive purging: MTTR, arrival counts and chronic-defect detection all continue to work on rows whose text has been redacted, because they depend on timestamps, domain and room — not on what the resident wrote.

11.5 Secrets

Secret	Handling
Google service-account JSON	Base64 in `GOOGLE_SA_KEY` env var. **Never a file in the repo.**
Database credentials	`DATABASE_URL` env var, injected by the platform
Session signing key	`APP_SECRET` env var, generated per environment
VAPID key pair	`VAPID_PUBLIC_KEY` / `VAPID_PRIVATE_KEY` env vars. The private key authenticates us to push services; rotating it invalidates every subscription, so it is generated once per environment

`.gitignore` blocks `**/google-service-account*.json`, `**/credentials.json`, `.env*` and `application-local.yml` (Sec. repo root). CI runs **Gitleaks** on every PR so a committed secret fails the build rather than reaching `main`. The Sheets service account is granted **read-only** access to the single response sheet — if it leaks, the blast radius is reading complaints we already have.

11.6 Application security

Control	Implementation
SQL injection	JPA/parameterised queries only; no string-concatenated SQL
XSS	Thymeleaf escapes by default; `th:utext` is banned and checked in review
CSRF	Spring Security tokens on all state-changing form posts
Clickjacking	`X-Frame-Options: DENY`
Transport	HTTPS enforced (`Strict-Transport-Security`); platform-terminated TLS
Headers	CSP, `X-Content-Type-Options: nosniff`, `Referrer-Policy`
Upload safety	CSV only, ≤5 MB, ≤5000 rows, content-type and header validated before parsing
Push payload privacy	Payloads are end-to-end encrypted (`aes128gcm`) per the Web Push spec, so Google's or Mozilla's push service relays ciphertext it cannot read. Bodies still avoid resident identifiers — a lock-screen preview is visible to anyone holding the phone, so notifications name a room, never a person
Dependencies	OWASP Dependency-Check in CI; Dependabot enabled
Error leakage	RFC 7807 responses only; stack traces never returned to a client

11.7 Audit as a security control

`audit_event` (Sec. 3.7) is append-only and covers authentication attempts, provisioning, every state transition, every point movement, `act-as` actions, and configuration changes.

Two properties make it trustworthy rather than decorative: it is written **in the same transaction** as the change it records, so it cannot silently diverge; and there is **no application code path that updates or deletes an audit row** — the JPA entity exposes no setters and the repository no delete method. Since the ledger is also append-only, an attender's monthly bonus can be reconstructed from first principles and defended line by line in a dispute, which is the situation this control actually exists for.

12. Test Plan

12.1 Strategy

Level	Tool	Scope	Target
Unit	JUnit 5, AssertJ	Pure domain logic — scoring, classification, state machine, statistics	~70% of tests; no Spring context
Slice	`@DataJpaTest`, `@WebMvcTest`	Repositories against real Postgres; controller contracts	~15%
Integration	`@SpringBootTest` + Testcontainers	Full pipeline, transactions, concurrency	~10%
Concurrency	JUnit + `CountDownLatch` / `ExecutorService`	The races in Sec. 7.1	Explicit, named suite
Architecture	ArchUnit	Layering rules (Sec. 2.4)	Continuous
Manual/UAT	Stakeholder	Acceptance criteria (Sec. 12.6)	Per milestone

All database tests run against real PostgreSQL via Testcontainers, never an in-memory substitute. The properties we most need to verify — unique-constraint behaviour under concurrent inserts, optimistic-lock failures, `SELECT ... FOR UPDATE` — are precisely the ones an H2 stand-in would fake convincingly and wrongly. Coverage floor: **75% line, 90% on `domain`**, enforced by JaCoCo in CI.

12.2 Fixtures

Fixture	Source	Size	Used by
F1 — Real anonymised complaints	Cauvery Hostel logs, PII replaced with dummy data	~200 rows	Classifier gold set, realistic dispatch
F2 — Synthetic edge cases	Hand-authored	~60 rows	Corner-case suite (Sec. 12.4)
F3 — Concurrency workload	Generated	4 attenders × 25 tasks	Race tests (Sec. 12.5)
F4 — Volume/telemetry	Generated, 90 days	~1800 complaints	Analytics, anomaly detectors
F5 — Malformed sheet	Hand-authored	~30 rows	Ingestion robustness

Anonymisation of F1 happens once, at extraction, before the data ever enters the repository. A committed fixture is effectively permanent, so the scrub cannot be a later step: emails → `resident{n}@example.com`, names removed, room numbers remapped through a fixed permutation that preserves floor structure (so dispatch tests stay meaningful) while breaking identifiability. The mapping table is not committed.

12.3 Categorization gold set

The proposal's "≥90% accuracy" needs a defensible methodology, not a spot check.

Construction. From F1, 200 complaints labelled by hand into `ELECTRICAL`, `CIVIL`, `RO_PLANT`, `NETWORK`, `AMBIGUOUS`. Split **70/30 into development and holdout**. Lexicon terms and thresholds are tuned *only* against development; the holdout is scored once per milestone. Tuning against the holdout would produce a number that means nothing.

Metrics. Reported together, never in isolation:

```
coverage  = auto-accepted / total
accuracy  = correct predictions / auto-accepted          ← the ≥90% criterion
recall_d  = correctly predicted as d / truly d
```

Plus a per-domain confusion matrix.

Coverage is reported alongside accuracy because **accuracy alone is trivially gameable**: a classifier that refuses to classify anything scores 100%. Targets: **accuracy ≥ 90%, coverage ≥ 70%** on the holdout. Complaints labelled `AMBIGUOUS` by the human labeller are expected to land in triage; if one is auto-accepted, that is counted as an error.

Continuous evaluation. Every manager triage correction in production is `(text, true label)` — a free growing labelled set. A monthly re-scoring job reports drift, and this corpus is also what would eventually make the OpenNLP stage (Sec. 5.7) viable.

12.4 Corner-case suite

Every case named in the proposal's verification plan, plus what the design surfaced:

#	Case	Expected
E1	Ambiguous text scoring two domains equally	`UNCLASSIFIED` → triage, both scores shown
E2	Identical simultaneous submissions (different residents)	Two complaints — distinct `response_key` (Sec. 5.3)
E3	Byte-identical resubmission by the same resident	One complaint — `response_key` collides
E4	Same CSV uploaded twice	Zero duplicates
E5	Unallocated / unparseable floor number	`TRIAGE` / `UNMAPPED_LOCATION`
E6	All attenders marked unavailable	`TRIAGE` / `NO_AVAILABLE_ATTENDER`, nothing lost
E7	Floor exists, nobody covers it	Assigned to least-loaded, `UNCOVERED_FLOOR` warning
E8	Every attender over `L_cap`	Assigned to best, `CapacityExceeded` notification
E9	Sheet row deleted mid-month	`VANISHED`, complaint survives, manager flagged
E10	Sheet row edited after ingestion	Amended in place, re-classified, no duplicate
E11	Row deleted causing all indices to shift	Zero spurious amendments (Sec. 5.1)
E12	Required column renamed	Run fails loudly with `INGESTION_COLUMN_MISSING`
E13	Empty / whitespace-only complaint text	`UNCLASSIFIED` → triage
E14	Tamil-transliterated text ("thanni varala")	Fuzzy/lexicon hit or triage — never a wrong confident domain
E15	Resolve attempted with no docket	`GUARD_FAILED` 422
E16	Docket failing the configured pattern	`INVALID_DOCKET` 422
E17	Docket corrected after entry	Old row `SUPERSEDED`, both retained
E18	Resolve → reopen → resolve again	Credit, reversal, new credit. Net +2. Three ledger rows
E19	Penalty backdated into a closed period	Applied to correct period, manager warned
E20	Attender A acts on attender B's task	403 `NOT_ASSIGNED`
E21	Month boundary at 02:00 IST on the 1st	Falls in the new month (NFR-7)

#	Case	Expected
E22	Sheets API down for 6 hours	Backoff, degraded banner, full catch-up on recovery
E23	Anomaly detector below its `n` gate	Emits nothing (Sec. 8.3)
E24	`MAD = 0` in A1	Falls back to `3 × median`, no divide-by-zero
E25	Attender queue with 0 tasks	Empty state, not an error
E26	Leaderboard with an attender at 0 points	Appears, ranked last (`LEFT JOIN`, Sec. 7.8)
E27	Same browser re-subscribes to push	Existing row updated, not duplicated (`uq_push_endpoint`)
E28	Push service returns 410 Gone	Subscription revoked; no further attempts
E29	Attender with no push subscription is assigned a task	Assignment succeeds; push skipped silently
E30	Push service unreachable for an hour	Retries then `FAILED`; **no state transition affected**
E31	Outbox poller re-runs after a crash	No duplicate push per device (`uq_push_delivery`)

12.5 Concurrency suite

Direct tests of Sec. 7.1. Each runs against real Postgres, using a `CountDownLatch` so threads start together, and asserts on **final database state** rather than on response codes.

#	Test	Setup	Assertion
X1	Double resolve, same key	10 threads, same task, same `Idempotency-Key`	Exactly 1 ledger row, +2 total, all 10 get 200
X2	Double resolve, different keys	10 threads, same task, distinct keys	Exactly 1 ledger row — `RESOLVE:{task}:{att}` collides regardless (Sec. 7.3)
X3	Reassign vs resolve	Manager reassigns while attender resolves	Exactly one succeeds; loser gets 409; ledger consistent with the winner
X4	Shared duty, concurrent completion	4 attenders, shared task, simultaneous	4 ledger rows (one each), task `COMPLETED` exactly once
X5	Over-claim	4 attenders claim a `k=1` duty at once	1 assignment; 3 receive `NO_SLOTS_LEFT`
X6	Concurrent ingestion	3 poll runs of the same rows in parallel	No duplicate complaints; ShedLock serialises
X7	Leaderboard under write load	Read while 100 credits are written	Every read internally consistent; final total exact
X8	Reopen race	Manager reopens while attender re-resolves	Net points correct; no orphaned credit
X9	**Proposal performance fixture**	4 attenders × 25 tasks resolved concurrently	100 ledger rows, 200 points, zero lost updates, p95 < 500 ms

X2 is the sharpest test in the suite: it proves that correctness does **not** depend on the client sending a good idempotency key. Even a buggy client generating a fresh UUID per retry cannot double-credit, because the ledger key is derived from the work, not from the request.

12.6 Acceptance-criteria traceability

Each stakeholder-facing criterion from proposal Sec. 8, mapped to what proves it:

Criterion	Verified by	Milestone
Ingests automatically, no copy-paste, no duplicates	E2, E3, E4, E9, E10, E11, X6 + live sheet demo	Mid-demo
Categorizes ≥90% into Civil/Electrical/RO	Sec. 12.3 holdout evaluation	Mid-demo
Assigns to the primary floor attender	Dispatch unit tests, E5–E8	Mid-demo
Attender queue on mobile: docket entry + resolve, +2 credited	E15–E18, X1, X2 + device walkthrough	Mid-demo
Manager logs −1 penalties and special duties	Ledger tests, E19	Final
Real-time leaderboard	Sec. 7.8 tests, X7, SSE integration test	Final
One-click clean `.xlsx` export	POI tests, manual inspection	Final
Anomaly detection, bottlenecks, visualizations	Sec. 8 detector tests on F4, E23, E24	Final
Attenders alerted on their phone when work is assigned (G8)	E27–E31 + device walkthrough on a real Android phone	Final

12.7 What we are not testing

Stated to keep the plan honest:

- **Browser matrix** — Chrome on Android and desktop only; that is what these five people use.
- **Load beyond NFR-1.** X9 is the stated performance fixture. We do not test 1000 concurrent users, because there will never be five.
- **Google Sheets API itself** — mocked at the port boundary (Sec. 5.8). We test our reconciliation logic, not Google's service.
- **Browser push services themselves** (FCM, Mozilla autopush) — mocked at the HTTP boundary. We test our subscription handling, payload construction and failure responses, not Google's delivery.
- **iOS Safari push** — attenders are on Android (Sec. 10.1).
- **The IITM EU CMS app** — no integration exists to test (Sec. 1.8).
- **Penetration testing** — out of scope per Sec. 11.1.

13. Revised Timeline

13.1 Fixed dates

Milestone	Date	Days from 8 Sep
Design document	**11 Sep 2026**	3
Mid-demo	**9 Oct 2026**	31
Final submission	**6 Nov 2026**	59

Solo developer, alongside coursework. Planned capacity **~15 hours/week**, so roughly 130 hours total across the build. Every estimate below is sized against that, not against a full-time week.

13.2 Sprint plan

```
gantt
    title Build plan — 8 Sep to 6 Nov 2026
    dateFormat YYYY-MM-DD
    axisFormat %d %b

    section Design
    Design doc + repo scaffold      :done, s0, 2026-09-08, 4d

    section Core pipeline
    Foundation, schema, auth        :s1, 2026-09-12, 7d
    Ingestion + categorization      :s2, 2026-09-19, 7d
    Dispatch + workflow + ledger    :s3, 2026-09-26, 7d
    Mobile + dashboard + deploy     :s4, 2026-10-03, 7d
    MID-DEMO                        :milestone, m1, 2026-10-09, 0d

    section Full system
    Duties, penalties, concurrency  :s5, 2026-10-10, 7d
    Analytics + anomaly detection   :s6, 2026-10-17, 7d
    Excel, SSE, offline, polish     :s7, 2026-10-24, 7d
    Hardening + UAT + sign-off      :s8, 2026-10-31, 7d
    FINAL                           :milestone, m2, 2026-11-06, 0d
```

13.3 Sprint detail

Each item is tagged **[C] Core** (must ship) or **[E] Extended** (first to be cut).

S0 · 8–11 Sep · Design & scaffold

- [C] This design document, Sec. 1–14
- [C] Repo scaffold, CI skeleton, PR workflow, branch protection
- [C] Email stakeholder with OQ-1 (docket format) and OQ-2 (SLA targets)

Exit: design doc merged via PR; CI green.

S1 · 12–18 Sep · Foundation

- [C] Maven project, Spring Boot 3.x, package layout per Sec. 2.3
- [C] Flyway migrations for the full Sec. 3 schema
- [C] JPA entities with `@Version`; repositories
- [C] Spring Security: roles, password + PIN, device tokens, lockout (Sec. 11.2)

- [C] Manager provisioning screen
- [C] Testcontainers Postgres in CI; ArchUnit layering rules

Exit: manager and 4 attenders can log in; schema migrates clean; CI runs integration tests.

S2 · 19–25 Sep · Ingestion & categorization

- [C] `ComplaintSource` port; Google Sheets adapter; header mapping (Sec. 5.2)
- [C] Reconciliation algorithm with `response_key` / `content_hash` (Sec. 5.3)
- [C] CSV upload adapter (Risk 1 Plan B)
- [C] Lexicon classifier with fuzzy matching; `category_term` seeding
- [C] Gold set F1 built, labelled, split; first accuracy measurement
- [C] Corner cases E1–E14
- [E] Manager lexicon editor UI

Exit: live sheet ingests with zero duplicates; holdout accuracy measured and reported.

S3 · 26 Sep – 2 Oct · Dispatch, workflow, ledger

- [C] Dispatch scoring, tie-breaks, degenerate cases, score snapshot (Sec. 6)
- [C] Workflow state machine + full transition table (Sec. 4)
- [C] Append-only ledger with idempotency keys (Sec. 7.3)
- [C] Audit event writing on every transition
- [C] Priority aging + SLA computation
- [C] Concurrency tests X1, X2, X3

Exit: ingest → classify → dispatch → resolve → +2 credited, end to end, provably once.

S4 · 3–9 Oct · Interfaces & deployment → MID-DEMO

- [C] Attender mobile queue and 4-screen resolution flow (Sec. 10.2)
- [C] Manager dashboard: counters, triage queue, complaint list
- [C] Docket entry and validation
- [C] Cloud deploy (.jar on Render/Railway) + managed Postgres
- [E] Offline service worker

Exit / mid-demo deliverable: automated ingestion with CSV fallback, rule-based categorization, priority aging, floor dispatch, attender mobile with docket + points, live manager task lists, deployed to cloud. **Matches the proposal's mid-demo commitment exactly.**

S5 · 10–16 Oct · Duties, penalties, concurrency

- [C] Special duties: post, claim, complete (Sec. 4.4)
- [C] Penalty interface (−1) with mandatory notes
- [C] Manual adjustments and reversals
- [C] Shared-task concurrency (Sec. 7.5), pessimistic claim (Sec. 7.6)
- [C] Leaderboard (Sec. 7.8)
- [C] Concurrency suite X4–X9 complete

Exit: full point lifecycle correct under concurrent load; X9 passes.

S6 · 17–23 Oct · Analytics

- [C] Stage-duration extraction from audit log
- [C] MTTR / median / p90, bottleneck identification (Sec. 8.2)
- [C] Detectors A1 (MAD outlier), A2 (chronic location), A3 (Poisson surge)
- [C] Chart.js visualizations
- [E] Detectors A4 (SLA cluster), A5 (workload imbalance)

Exit: dashboard shows real analytics over F4; detectors respect their `n` gates.

S7 · 24–30 Oct · Export, real-time, polish

- [C] Apache POI multi-tab `.xlsx` daily + weekly export
- [C] SSE with outbox replay (Sec. 9.5)
- [C] Assisted admin log / `act-as` (Risk 3 Plan B)
- [C] Accessibility and mobile performance pass (Sec. 10.5–10.6)
- [C] **Web Push to attender phones** (Sec. 9.6): VAPID setup, subscription endpoints, service-worker handler, notification settings, delivery-failure handling
- [E] Offline queue if deferred from S4
- [E] OpenNLP second stage (Sec. 5.7)

Exit: one-click export produces a clean workbook; dashboard updates live; a P1 assignment buzzes the attender's phone within 30 s.

S8 · 31 Oct – 6 Nov · Hardening & sign-off

- [C] Full test pass; coverage gates met
- [C] Final gold-set evaluation on holdout
- [C] Security review (Sec. 11 checklist), Gitleaks + Dependency-Check clean
- [C] Stakeholder UAT session against Sec. 12.6
- [C] README, runbook, viva preparation notes
- [C] **Stakeholder sign-off**

13.4 Change from the original proposal

Change	Reason
Postgres everywhere; SQLite dropped	SQLite serialises writers and cannot demonstrate the concurrency control that is a headline feature (ADR-001)
OpenNLP moved to Extended, rules promoted to primary	Training data availability is unproven; rules need none and are explainable (ADR-004)
Auth added as explicit S1 scope	Absent from the proposal, but the schema, role matrix and deployment all depend on it
SSE chosen over WebSocket	One-directional need; simpler; survives PaaS proxies (ADR-003)
A4/A5 detectors marked Extended	Both need ≥4 weeks of history to emit anything, so they cannot be demonstrated before the final date anyway
Push notifications to attenders added (G8, Sec. 9.6)	Stakeholder requirement added after the proposal. Delivered via Web Push on the service worker the offline queue already needs, so the marginal cost is small

13.5 Risk buffer and cut lines

There is **no slack** in the schedule; a lost week must come out of scope, not out of quality. The cut order is fixed in advance so the decision is not made under pressure:

1. OpenNLP second stage (Sec. 5.7)
2. Detectors A4, A5
3. Offline service worker *replay* (degrades to online-only; idempotency keys still protect retries). The service-worker **registration** stays, because push depends on it.
4. Manager lexicon editor UI (seed via migration instead)
5. Weekly export (keep daily)

Never cut, because each is load-bearing for a stakeholder acceptance criterion or a core integrity guarantee: ingestion idempotency, the ledger and its constraints, the state machine, dispatch, the attender mobile flow, push notification of new assignments, penalties, the leaderboard, daily Excel export.

Risk	Trigger	Response
Sheets API access not granted in time	No credentials by 19 Sep	Build against a fixture sheet on a personal account; CSV path is unaffected
Real complaint data unavailable for F1	Nothing by 19 Sep	Generate F2-style synthetic corpus; report gold-set results with that caveat stated
Categorization misses 90%	Holdout below target at S2 exit	Expand lexicon from misses; raise τ (trading coverage for accuracy — accuracy is the stated criterion)
Free-tier deploy proves unusable	Failure during S4	Local-server fallback (Sec. 2.6) is already a designed deliverable, not a new build
Coursework crunch	Any sprint slips >3 days	Apply the cut list above, in order

14. Decision Record

Architecture Decision Records for the choices that shaped this design. Each states what was decided, what it cost, and what would make us revisit it.

ADR-001 · PostgreSQL everywhere, not SQLite/H2 for development

Status: Accepted · 8 Sep 2026

Context. The proposal specified SQLite or embedded H2 for local development with PostgreSQL in production. That is a common and convenient split.

Decision. PostgreSQL in development (Docker Compose), test (Testcontainers) and production. H2 is retained *only* for the offline fallback jar (Sec. 2.6).

Rationale. - SQLite serialises writers. The shared-task concurrency control (Sec. 7) is a headline feature of this project; on SQLite the tests would pass for the wrong reason and prove nothing about production. - H2 and Postgres diverge on constraint-violation behaviour, isolation semantics and locking — exactly the surface our correctness argument (Sec. 7.3) rests on. - Dialect drift in JPA surfaces late and expensively.

Consequences. Requires Docker locally. Schema is constrained to the portable SQL subset (Sec. 3.8) so the fallback jar still runs. Accepted: the constraint costs a few Postgres-only conveniences and buys tests that mean what they say.

Revisit if: the offline fallback becomes the primary deployment.

ADR-002 · Local credentials, invite-only; no Google OAuth

Status: Accepted · 8 Sep 2026

Context. Users are one manager and ~4 attenders. Requirements: people without institute accounts must be able to use the system, but only a specific pre-registered set may have access.

Decision. No self-registration. Every principal is a pre-provisioned `app_user` row. Manager authenticates with username + password; attenders with a **per-user** 6-digit PIN plus a device-bound persistent token. No OAuth of any kind.

Rationale. - **Decisive:** the proposal requires an offline local-server fallback for outages (Sec. 2.6). Google OAuth cannot complete a redirect flow without internet — it would fail during precisely the outage the fallback exists for. - Local credentials require no institute account, so "non-institute people must have access" is met by *removing* an external dependency rather than adding one. - "Only a specific set registered" is really the invariant *no self-registration*, which holds independently of credential type. - OAuth would add a consent screen, a public redirect URI, extra secrets and a second login path to test — for one user's convenience.

Consequences. We store password and PIN hashes, so we own credential hygiene: BCrypt cost 12, lockout, rate limiting, audited attempts (Sec. 11.2). A 6-digit PIN is weak in isolation; the design states this openly and compensates with lockout, no username enumeration, a small blast radius per attender, and one-click device revocation.

Revisit if: the user population grows beyond ~20, or the institute mandates SSO.

ADR-003 · SSE, not WebSocket, for live updates

Status: Accepted · 8 Sep 2026

Context. The proposal listed "Spring WebSocket / SSE" for real-time manager notifications.

Decision. Server-Sent Events, with a transactional outbox for replay.

Rationale. The requirement is strictly one-directional — server pushes duty updates to one manager dashboard. SSE is plain HTTP: it traverses PaaS proxies cleanly, reconnects automatically, and needs no STOMP broker or fallback negotiation. Its native `Last-Event-ID` mechanism pairs exactly with an outbox table (Sec. 3.7) to give gap-free replay after the free tier drops an idle connection.

Consequences. No client→server push channel; state changes go over normal HTTP POSTs, which is what we wanted anyway since they must pass through the state machine. SSE is limited to ~6 concurrent connections per browser origin — irrelevant at one manager.

Revisit if: bidirectional realtime is ever needed (e.g. live attender-to-manager chat).

ADR-004 · Rule-based lexicon as the primary classifier; OpenNLP deferred

Status: Accepted · 8 Sep 2026

Context. The proposal names Apache OpenNLP for "rule-assisted text categorization", with an acceptance criterion of ≥90% accuracy.

Decision. A weighted lexicon with fuzzy matching and a confidence gate is the primary and only required classifier. OpenNLP `DocumentCategorizer` is designed in as an optional second stage, consulted only for items the rules route to triage, and is tagged Extended (Sec. 13.3).

Rationale. - A supervised model needs a few hundred labelled examples per class. We have no confirmation that volume of labelled data exists, and the project cannot depend on an unverified input. - Complaint text is short, misspelled and often code-mixed Tamil/English transliteration — where lexicon + edit-distance degrades gracefully and a thin-data model does not. - The lexicon is editable by the manager without a redeploy. - Every decision is explainable by naming the terms that fired — which matters for the viva and for the manager's trust in the triage card (Sec. 10.3).

Consequences. Requires curating a domain lexicon (a few hours, with the stakeholder). Ceiling on accuracy is lower than a well-trained model's — mitigated by the confidence gate routing uncertain items to triage rather than guessing. Production triage corrections accumulate the labelled corpus that would make stage two viable later, so nothing is foreclosed.

Revisit if: holdout accuracy stalls below 90% with coverage above 70%.

ADR-005 · Greedy per-complaint dispatch, not global optimisation

Status: Accepted · 8 Sep 2026

Context. Assignment could be formulated as a global optimisation (Hungarian algorithm, min-cost flow) over a batch of complaints and attenders.

Decision. Greedy `argmax` of a weighted score, per complaint, with a deterministic tie-break chain (Sec. 6.2).

Rationale. Complaints arrive one at a time from a 3-minute poll; a typical batch is 1–3 items, and the optimal assignment of a 1-item batch *is* the greedy one. With $|A| = 4$ the theoretical gain is negligible, while batch optimisation would require holding complaints back to accumulate a batch — adding latency to exactly the P1 cases that matter most. Greedy is also explainable, and "Ravi 0.78 vs Kumar 0.50" is something a manager can challenge.

Consequences. No global optimality guarantee. Anti-starvation must be handled explicitly, which it is, via the tie-break chain and the load term.

Revisit if: attender count grows past ~15, or complaints start arriving in large batches.

ADR-006 · Pessimistic locking for duty claims, optimistic elsewhere

Status: Accepted · 8 Sep 2026

Context. Claiming a special-duty slot is a read-modify-write on a count (`active < k`). The house style everywhere else is optimistic locking.

Decision. `SELECT ... FOR UPDATE` on the task row for claims only.

Rationale. Optimistic locking would work correctly, but its user-visible failure is a retry prompt for someone standing in a corridor. Contention here is trivially low — 4 people, a handful of duties a week — so a lock held for microseconds costs nothing and lets the loser get an immediate, unambiguous "already claimed". The clarity is worth the inconsistency in style, and the inconsistency is documented rather than accidental.

Consequences. One deliberate deviation from the OCC default. `uq_task_attender` provides a second, independent guarantee against self-double-claim.

Revisit if: claim contention ever becomes non-trivial.

ADR-007 · `READ COMMITTED` isolation, not `SERIALIZABLE`

Status: Accepted · 8 Sep 2026

Context. The proposal calls for "strict transactional isolation". The strongest reading is `SERIALIZABLE`.

Decision. PostgreSQL's default `READ COMMITTED`, with correctness carried by uniqueness constraints and version checks.

Rationale. Every invariant we depend on — no double-credit, no lost update, no over-claim — is enforced by a `UNIQUE` constraint, an `@Version` check, or a row lock. All three hold at `READ COMMITTED`. `SERIALIZABLE` would add serialization failures and retry complexity across every transaction to buy a guarantee already obtained more cheaply and more locally.

Consequences. Non-repeatable reads are possible within a transaction; no invariant relies on their absence. Any *future* invariant depending on a multi-row consistent snapshot would need explicit locking or a stronger level for that transaction alone.

Revisit if: a new requirement needs cross-row consistency that constraints cannot express.

ADR-008 · Append-only ledger; no stored point balance

Status: Accepted · 8 Sep 2026

Context. The obvious implementation of a point total is a counter column on the attender.

Decision. `point_ledger` is append-only. Balances are `SUM` aggregates. Corrections are compensating rows, never updates or deletes. No `attender.points` column exists.

Rationale. - A counter has a lost-update race on every increment; an insert with a unique key does not. - Bonus disputes require answering "why is this number 34?" — a ledger answers line by line, a counter cannot answer at all. - Reversal-by-compensation keeps the reopen cycle (T17) auditable. - At NFR-1 volume the aggregate is an index scan over a few hundred rows; there is no performance argument for denormalising.

Consequences. Every point read is an aggregate query. Accepted, and measured: ~600 rows per period. If volume ever justified it, the right answer is a materialized view refreshed on commit — never a mutable counter, which would give back the property that makes this design correct.

Revisit if: ledger volume exceeds ~1M rows.

ADR-009 · Content-derived identity for source rows, not row index

Status: Accepted · 8 Sep 2026

Context. Ingestion must detect new, edited and deleted Google Form responses without a change feed.

Decision. Identity is `response_key = SHA256(timestamp || email)`; change detection is a separate `content_hash` over all cells. Row index is stored as a positional hint only.

Rationale. Deleting one sheet row shifts every index below it. Keying on `(sheet, row_index)` would report the entire tail as amended after a single deletion, and — depending on the reconciliation rule — could duplicate hundreds of complaints in one poll. A Google Form response retains its submission timestamp and submitter across edits, so a content-derived key is stable across both row shifts and edits. Separating identity from change detection is what lets one algorithm handle inserts, amendments and deletions coherently.

Consequences. Requires the Form to collect email, or timestamp-only keys with an in-batch disambiguator for same-second submissions (Sec. 5.3). Deletion can only be concluded from a **full** scan, never an incremental one — the most important guard in the algorithm.

Revisit if: the Form is replaced by a source with stable native row IDs.

ADR-010 · Modular monolith, not microservices

Status: Accepted · 8 Sep 2026

Context. The proposal describes several distinct subsystems (ingestion, dispatch, ledger, analytics) that could be separate services.

Decision. One Spring Boot jar with enforced internal module boundaries (Sec. 2.3–2.4), ports and adapters only where an external system is involved.

Rationale. At ≤50 complaints/day with 4 users, distribution would add network failure modes, deployment complexity and eventual-consistency problems in exchange for scale we will never need. Critically, our core correctness guarantees (Sec. 7) rely on **single-database transactions**; splitting the ledger from the workflow would turn a uniqueness constraint into a distributed-transaction problem. Module boundaries are drawn so a component *could* be extracted if that ever changed.

Consequences. One deployment unit, one database, one failure domain. ArchUnit enforces the boundaries that would otherwise erode (Sec. 2.4).

Revisit if: the system is ever extended to multiple hostels with independent operations.

ADR-011 · Web Push with VAPID for attender notifications, not FCM or a native app

Status: Accepted · 8 Sep 2026

Context. Attenders must be alerted on their phone when work is assigned, without having to open the app and look (G8). This was added after the proposal, which had listed manager-side SSE only.

Decision. The **Web Push protocol with VAPID**, delivered to the service worker the PWA already registers for offline replay. No Firebase project, no Google Play binary, no third-party push SDK. Library: `nl.martijndwars:web-push`.

Rationale. - The service worker is **already required** for the offline action queue (Sec. 10.2). Push reuses that registration, so the marginal cost is a subscription table, an endpoint pair and a message handler — not a new client platform. - VAPID needs only a self-generated key pair. FCM would mean a Firebase project, a Google account dependency and an SDK, to reach the same Android Chrome that speaks Web Push natively. - A native app would need Play Store distribution, signing keys and an update path for four people who already use the web app — plainly disproportionate. - Payloads are end-to-end encrypted, so the relaying push service sees only ciphertext.

Consequences. - Attenders must grant browser notification permission, and a denial is not recoverable from inside the app. Mitigation: ask only *after* first successful login, once the app has shown its value (Sec. 10.2), and expose a settings toggle so people dial alerts down rather than revoking them. - Push **cannot work on the offline fallback jar** (Sec. 2.6) — there is no internet to reach a push service. Acceptable: during an outage the manager is running roll-call in person anyway, which is the assisted-admin path (Sec. 9.4). - iOS would require an installed PWA on Safari 16.4+. Out of scope; attenders are on Android. - Delivery is best-effort and no state transition depends on it (NFR-10). The in-app queue stays the source of truth, so a missed push degrades convenience, never correctness.

Revisit if: attenders move to iOS in numbers, or delivery reliability proves inadequate in the field — at which point FCM's data channel would be the next step, not a native app.